

Playwright Interview Preparation Handbook (2026 Edition)

Volume 1 - Architecture, Fixtures, Auth, Locators, Sync, Parallelism, Config, API, Network Mocking, Debugging

Prepared for Srikanth, SDET and Associate Architect QE

September 2026

Contents

How to Use This Handbook	4
SECTION 1 — Playwright Architecture & Framework Design (Senior Level)	5
Q1. Explain the Browser → BrowserContext → Page hierarchy, and why this matters for enterprise test suites.	5
Q2. Why does BrowserContext isolation matter specifically for parallel enterprise suites, and what breaks if you get it wrong?	6
Q3. Design a Playwright framework folder structure for a suite that will grow past 1,000 tests. Justify each top-level folder.	7
Q4. Monorepo with one Playwright project vs. multiple separate Playwright projects — how do you decide?	8
Q5. What is storageState-based authentication architecture, and how do you design it for multiple user roles across a large suite?	9
Q6. How do you architect authentication for SSO (Okta/Azure AD) in Playwright, given the UI redirect flow is often unreliable for automation?	10
Q7. How would you design multi-context testing to simulate two users interacting in real time (e.g., a shared dashboard, chat, or collaborative report)?	11
Q8. What does “dependency injection using fixtures” mean architecturally in Playwright, and why is it a better pattern than helper functions imported into every spec file?	12
SECTION 2 — Advanced Fixtures (Most Asked in 2026)	14
Q9. Explain test-scoped vs. worker-scoped fixtures, and give a real scenario where getting the scope wrong causes a production-suite bug.	14
Q10. What is fixture composition, and how do you build a fixture that depends on multiple other fixtures without creating tangled setup order?	15
Q11. What are “lazy fixtures,” and when should a fixture’s setup cost not be paid unless a test actually uses it?	16
Q12. Walk through fixture teardown guarantees — what happens if the test body throws an exception mid-test? Does teardown still run?	16
Q13. How would you design a worker fixture that provides pre-authenticated users for multiple roles, without every worker re-doing the login flow?	17
Q14. Name two or three fixture anti-patterns you’ve seen in real codebases, and how you’d refactor them.	18

SECTION 3 — Authentication (Extremely Important)	19
Q15. Compare API-based login vs. UI-based login for test setup, and explain when you'd deliberately still use UI login.	19
Q16. How do you handle token expiry and session timeout scenarios in a long-running regression suite (e.g., 2+ hour run, 1-hour token TTL)?	20
Q17. Design a parallel login strategy for a suite where the auth provider rate-limits login attempts.	21
Q18. How would you test a multi-tenant application where each tenant has separate SSO configuration?	21
SECTION 4 — Locator Strategy (Modern Playwright)	22
Q19. Explain locator strict mode and why it's a deliberate design choice, not a limitation.	22
Q20. Why does Playwright recommend getByRole as the top locator priority, and when do you deviate from it?	23
Q21. How do you locate elements inside a virtualized (windowed) table where most rows aren't in the DOM at any given time?	23
Q22. How do you write stable locators into Shadow DOM components, and where does Playwright's piercing behavior stop?	24
Q23. A locator works locally but is flaky in CI only. Walk through your debugging process.	25
SECTION 5 — Auto Waiting & Synchronization	25
Q24. Explain Playwright's actionability checks. What exactly does the framework verify before performing a click?	25
Q25. What's the difference between "actionability" and "business readiness," and why does confusing them cause flaky tests?	26
Q26. Why is page.waitForTimeout() considered bad practice, and what should replace it in each common use case?	27
Q27. How do you reliably wait for content rendered via React Suspense / lazy-loaded chunks?	27
Q28. How do you test a feature backed by background polling or long-polling, where the UI updates asynchronously without a page navigation?	28
SECTION 6 — Parallel Execution (Top Interview Topic)	29
Q29. Explain the difference between workers and shards, and when you'd scale one vs. the other.	29
Q30. What is fullyParallel, and how does it differ from the default file-level parallelism?	29
Q31. When and why would you use test.describe.serial, and what's the trade-off?	30
Q32. Two tests in different files unexpectedly share state and cause flaky parallel failures. How do you diagnose the shared-data source?	31
SECTION 7 — Playwright Configuration (playwright.config.ts)	32
Q33. How do you structure playwright.config.ts to support multiple environments (dev, staging, prod-smoke) cleanly?	32
Q34. Explain how Playwright merges use options across the global config, project-level config, and test.use() in a spec file. What wins?	33
Q35. How do you configure expect timeouts, retries, and snapshot comparison settings differently for a visual-regression-heavy project vs. a functional one? . . .	34
SECTION 8 — API Testing (Advanced)	34
Q36. How do you architect a reusable, typed API client on top of APIRequestContext for hybrid API+UI testing?	34
Q37. How do you test chained/dependent API calls, where call B needs an ID returned by call A?	35

Q38. How do you validate an API response against a JSON schema instead of asserting individual fields one by one?	36
Q39. How do you handle multipart file upload testing through the API layer, and separately through the UI?	37
SECTION 9 — Network Interception & Mocking	38
Q40. Explain <code>page.route()</code> and the difference between <code>fulfill()</code> , <code>abort()</code> , and <code>continue()</code> . When do you use each?	38
Q41. How do you use HAR files for network replay, and when is that better than live <code>page.route()</code> mocking?	39
Q42. How do you test a feature flag that's controlled server-side, without waiting for the actual flag to be toggled in a real environment?	40
Q43. How would you simulate slow or degraded network conditions to test a loading/timeout UX, without actually throttling the real network?	40
SECTION 10 — Debugging & Flaky Test Analysis	41
Q44. Walk through your systematic process for debugging a test that fails only in CI, never locally.	41
Q45. What information does the Trace Viewer give you that a screenshot alone cannot?	42
Q46. A test passes when run alone but fails when run as part of the full suite. What's your hypothesis-driven debugging approach?	43
RAPID-FIRE REVISION BANK	43
Page Object Model & Enterprise Architecture	43
CI/CD	44
Performance & Stability	44
Real Coding Interview Challenges (concise prompts)	45
QA Architect Scenario Questions (concise prompts)	45
AI + Playwright (2026)	46
Company-Specific Emphasis Patterns (from public interview reports, 2025–2026 cycles)	46

How to Use This Handbook

This is **Volume 1** of a two-volume Playwright interview prep series, built for Senior SDET / QA Architect / GenAI QE interviews (2025–2026 cycle, product companies and GCCs). Given the depth requested per question (difficulty, frequency, interviewer intent, spoken answer, healthcare-analytics example, TypeScript code, mistakes, follow-ups, architect tip), a genuinely useful single document covers **10 high-weight sections in full depth** rather than 20 sections diluted. A **Rapid-Fire Revision Bank** at the end gives quick-recall coverage across the remaining topics (POM, enterprise framework design, CI/CD, performance, coding challenges, architect scenarios, AI+Playwright, company-specific patterns).

Volume 2 (POM & Enterprise Framework Design, CI/CD, Performance, Coding Challenges, QA Architect Scenarios, AI+Playwright, Company-Specific Deep Dives, Debugging Case Studies) can be generated as a follow-up in the same format on request.

Frequency legend: ★★★★★ Very Frequently Asked · ★★★★ Frequently Asked · ★★★ Occasionally Asked · ★★ Rare but High-Value

Domain examples throughout use a healthcare-analytics platform context (dashboards, cohorts, visualizations, filters, reports, exports) similar to Artemis Health, per your current project background.

SECTION 1 — Playwright Architecture & Framework Design (Senior Level)

Q1. Explain the Browser → BrowserContext → Page hierarchy, and why this matters for enterprise test suites.

Difficulty: Senior **Asked In:** Microsoft, Walmart Global Tech, ServiceNow, Atlassian **Frequency:** ★★★★★

Why Interviewer Asks This This separates candidates who “use” Playwright from candidates who understand *why* it’s fast and isolated compared to Selenium. It also tests whether you can reason about resource cost at scale (workers × contexts).

Best Interview Answer A Browser is a single OS-level browser process — expensive to start, so we launch it once per worker and reuse it. A BrowserContext is a lightweight, isolated “incognito profile” inside that browser — its own cookies, storage, cache, and permissions — created in milliseconds. A Page is a tab inside a context. The key architectural insight is that **test isolation happens at the context level, not the browser level**. In an enterprise suite, you never share a context between tests, but you *do* share the browser process across many contexts within a worker, which is what makes Playwright dramatically cheaper than spinning up a new WebDriver session per test.

Real Project Example On a healthcare-analytics platform like Artemis Health, different user roles (analyst, admin, read-only viewer) need different storage states. Instead of restarting the browser per role, we launch one browser per worker and spawn a fresh context per test, injecting the right storageState for that role — a full login flow avoided per test while cookies/session never leak across tests.

TypeScript Code Example

```
import { chromium, Browser, BrowserContext, Page } from '@playwright/test';

let browser: Browser;

async function runIsolatedTest() {
  browser = await chromium.launch(); // one process, reused across contexts

  const analystContext: BrowserContext = await browser.newContext({
    storageState: 'auth/analyst.json',
  });
  const adminContext: BrowserContext = await browser.newContext({
    storageState: 'auth/admin.json',
  });

  const analystPage: Page = await analystContext.newPage();
  const adminPage: Page = await adminContext.newPage();

  await analystPage.goto('/dashboards/cohort-explorer');
  await adminPage.goto('/admin/user-management');

  // Cookies, localStorage, and cache in analystContext are fully
  // invisible to adminContext – no cross-contamination.

  await analystContext.close();
  await adminContext.close();
  await browser.close();
}
```

```
}
```

Common Mistakes Candidates say “a context is like a new browser” without mentioning it shares the underlying process — missing the performance angle. Others don’t know that permissions, geolocation, and viewport are context-scoped, not page-scoped, and give wrong answers when asked about multi-tab scenarios.

Follow-Up Questions 1. What’s shared between two contexts in the same browser, and what’s isolated? 2. How does this differ from Selenium’s session model? 3. When would you deliberately share a context across two pages? 4. What happens to open pages when you close a context vs. close a browser? 5. How would you test a “user opens the export in a new tab” flow?

QA Architect Tip Senior candidates connect this to cost: “one browser process per worker, N contexts per worker” is the sizing model they use when justifying worker/shard counts to engineering leadership — not just a trivia answer.

Q2. Why does BrowserContext isolation matter specifically for parallel enterprise suites, and what breaks if you get it wrong?

Difficulty: Senior **Asked In:** JP Morgan Chase, Salesforce, EPAM **Frequency:** ★★★★★

Why Interviewer Asks This Tests whether you’ve actually debugged flaky parallel runs, not just read documentation.

Best Interview Answer If two tests share a context (or worse, a page), state leaks: cookies set by test A’s login are visible to test B, localStorage flags persist, and one test’s network route mocks apply to the other. In parallel execution, this shows up as intermittent, hard-to-reproduce failures because the outcome depends on run order and worker scheduling. The fix is one context per test (which `test()` gives you by default in Playwright Test), and being deliberate about anything you intentionally want to persist, like an authenticated storage state file, which should be *read*, not *shared live*.

Real Project Example In a dashboard reporting module, a mocked “cohort size” API response left in place by a previous test caused a *different* test’s assertion on record counts to pass for the wrong reason — a false positive, which is more dangerous than a false failure.

TypeScript Code Example

```
import { test, expect } from '@playwright/test';

// Playwright Test gives each test() a fresh context + page automatically.
test('cohort count reflects applied filters', async ({ page }) => {
  await page.goto('/dashboards/cohort-explorer');
  await page.getByLabel('Age range').fill('40-60');
  await expect(page.getByTestId('cohort-count')).toHaveText(/d+ members/);
});

test('export respects active filters', async ({ page }) => {
  // Fresh context – no leftover route mocks or cookies from the test above.
  await page.goto('/dashboards/cohort-explorer');
  await page.getByRole('button', { name: 'Export CSV' }).click();
  await expect(page.getByText('Export queued')).toBeVisible();
});
```

Common Mistakes Manually creating a shared `browser.newContext()` in a `beforeAll` hook to “save time,” then wondering why tests fail only in CI under parallelism.

Follow-Up Questions 1. How would you detect a context-leak bug from CI logs alone? 2. What’s the risk of reusing a page across tests in the same file? 3. Where does `test.describe.serial` fit into this discussion?

QA Architect Tip Architects frame this as a “blast radius” problem — isolation limits how far one bad test can propagate failure into others.

Q3. Design a Playwright framework folder structure for a suite that will grow past 1,000 tests. Justify each top-level folder.

Difficulty: Staff / QA Architect **Asked In:** Microsoft, Walmart Global Tech, Deloitte Digital **Frequency:** ★★★★★

Why Interviewer Asks This This is a direct proxy for “have you actually run a large suite in production,” since folder structure decisions only get made painfully at scale.

Best Interview Answer I organize by *responsibility*, not by feature, at the top level: `tests/` for spec files (mirroring product IE areas), `pages/` for Page Object / component classes, `fixtures/` for custom test fixtures, `api/` for typed API clients used both standalone and inside UI tests, `utils/` for pure helper functions (date formatting, data builders), `config/` for environment-specific `playwright.config.ts` overlays, `test-data/` for fixtures/JSON, and `.auth/` for generated storage states (gitignored). The critical decision is keeping **business logic out of spec files** — a spec file should read like a user story, with implementation detail delegated to page objects and fixtures.

Real Project Example On the Artemis Health migration from Cypress, the biggest win was splitting `pages/dashboards/` into per-widget component objects (`FilterPanel`, `CohortTable`, `ExportModal`) instead of one giant `DashboardPage` class — this cut merge conflicts almost entirely once four engineers were writing tests in parallel.

TypeScript Code Example

```
project-root/
├── tests/
│   ├── dashboards/
│   │   ├── cohort-explorer.spec.ts
│   │   └── export.spec.ts
│   └── admin/
│       └── user-management.spec.ts
├── pages/
│   ├── dashboards/
│   │   ├── FilterPanel.ts
│   │   ├── CohortTable.ts
│   │   └── ExportModal.ts
│   └── BasePage.ts
├── fixtures/
│   ├── auth.fixture.ts
│   └── api-client.fixture.ts
├── api/
│   └── CohortApiClient.ts
├── utils/
│   └── dateBuilders.ts
```

```

├── config/
│   ├── playwright.config.ts
│   ├── playwright.staging.config.ts
│   └── playwright.prod-smoke.config.ts
├── test-data/
│   └── cohorts.json
└── .auth/                # generated at runtime, gitignored

```

Common Mistakes Mirroring the front-end folder structure exactly instead of test responsibility; putting assertions inside page objects (page objects should *expose state*, tests should *assert*).

Follow-Up Questions 1. Where do shared assertions/custom matchers live in this structure? 2. How do you prevent pages/ from becoming a dumping ground? 3. How does this structure change for a monorepo with multiple front-end apps? 4. How do you version test-data fixtures alongside API contract changes?

QA Architect Tip Staff-level answers mention *ownership boundaries* — which folders a feature team can touch without review from the platform QA team — because at 1,000+ tests, governance matters as much as structure.

Q4. Monorepo with one Playwright project vs. multiple separate Playwright projects — how do you decide?

Difficulty: Staff / QA Architect **Asked In:** ServiceNow, Atlassian, Razorpay **Frequency:** ★★★★★

Why Interviewer Asks This Tests architectural judgment on trade-offs rather than a “correct” answer — there isn’t one.

Best Interview Answer I use Playwright’s built-in projects array inside one config when the applications share auth, environment, and a release train — this gives one command, one report, shared fixtures, and per-project browser/device targeting. I split into genuinely separate Playwright installations (separate package.json, separate CI pipeline) when applications have independent release cycles, different ownership teams, or materially different dependency versions, because forcing them into one project creates a false coupling — a dependency bump for app A now blocks app B’s pipeline.

Real Project Example A single healthcare platform with a public marketing site and an authenticated analytics dashboard: same repo, same projects array, different testDir and different auth setup per project — but they ship together, so one config is correct. If the marketing site were a separate Next.js app with its own deploy cadence, I’d split it out.

TypeScript Code Example

```

// playwright.config.ts — one repo, two logical suites, shared infra
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  projects: [
    {
      name: 'dashboard-chromium',
      testDir: './tests/dashboards',
      use: { ...devices['Desktop Chrome'], storageState: '.auth/analyst.json' },
    },
    {

```

```

    name: 'admin-chromium',
    testDir: './tests/admin',
    use: { ...devices['Desktop Chrome'], storageState: '.auth/admin.json' },
  },
  {
    name: 'marketing-smoke',
    testDir: './tests/marketing',
    use: { ...devices['Desktop Chrome'] }, // no auth needed
  },
],
});

```

Common Mistakes Treating projects as only a browser-matrix feature and missing that it's also the right tool for auth/environment segmentation within one repo.

Follow-Up Questions 1. How do dependency updates get coordinated across split repos? 2. How do you share a Page Object between two separate Playwright installations? 3. What CI implications come from splitting vs. merging?

QA Architect Tip The strongest signal here is mentioning **team topology**, not just technical structure — architecture decisions in QA tooling usually mirror org boundaries (Conway's Law), and interviewers at Staff level are listening for that connection.

Q5. What is storageState-based authentication architecture, and how do you design it for multiple user roles across a large suite?

Difficulty: Senior **Asked In:** Walmart Global Tech, JP Morgan, Optum **Frequency:** ★★★★★

Why Interviewer Asks This Auth is the single most common source of flakiness and slowness in enterprise suites; this question filters for people who've solved it for real, not just followed a tutorial once.

Best Interview Answer storageState captures cookies and localStorage/sessionStorage after a successful login and serializes it to JSON. Instead of UI-logging-in for every test (slow, flaky, rate-limited), a one-time globalSetup (or a dedicated setup project) performs login via API or UI once per role, saves state files, and every test's context is created *with* that state already injected — so tests start already authenticated. For multiple roles, I generate one JSON file per role and map fixtures to the right file, rather than one shared “admin can do everything” user, because that hides role-based access bugs.

Real Project Example For Artemis Health-style dashboards, I maintain analyst.json, admin.json, and read-only.json storage states, refreshed nightly via a scheduled CI job before the token's TTL expires, so daytime test runs never pay the login cost or risk a login-flow regression blocking unrelated tests.

TypeScript Code Example

```

// global-setup.ts
import { request, FullConfig } from '@playwright/test';

async function globalSetup(config: FullConfig) {
  const roles = [
    { name: 'analyst', user: process.env.ANALYST_USER, pass: process.env.ANALYST_PASS },
    { name: 'admin', user: process.env.ADMIN_USER, pass: process.env.ADMIN_PASS },
  ];
}

```

```

for (const role of roles) {
  const context = await request.newContext();
  const response = await context.post('/api/auth/login', {
    data: { username: role.user, password: role.pass },
  });
  const { token } = await response.json();

  await context.storageState({ path: `./auth/${role.name}.json` });
  await context.dispose();
}
}

export default globalSetup;

// playwright.config.ts
export default defineConfig({
  globalSetup: require.resolve('./global-setup'),
  projects: [
    { name: 'analyst', use: { storageState: './auth/analyst.json' } },
    { name: 'admin', use: { storageState: './auth/admin.json' } },
  ],
});

```

Common Mistakes Logging in through the UI inside globalSetup when an API login endpoint exists (slower, more brittle); using one shared super-admin state for all tests, which masks permission bugs; forgetting the state file goes stale when the token TTL is shorter than the CI schedule.

Follow-Up Questions 1. What do you do when login requires MFA/OTP? 2. How do you handle token expiry mid-suite on a long-running regression run? 3. How would you test the login flow itself if all other tests skip it via storageState? 4. How do you keep secrets out of committed storageState files?

QA Architect Tip Architects always mention **the login flow still needs its own dedicated tests** that intentionally don't use storageState — bypassing auth everywhere means you've stopped testing auth.

Q6. How do you architect authentication for SSO (Okta/Azure AD) in Playwright, given the UI redirect flow is often unreliable for automation?

Difficulty: Senior / Staff **Asked In:** Microsoft, Deloitte Digital, Adobe **Frequency:** ★★★★★

Why Interviewer Asks This SSO is where most Selenium-era frameworks broke down entirely; this checks if you know the modern bypass pattern.

Best Interview Answer Where possible, I avoid driving the SSO UI redirect in every test entirely. I authenticate once against the identity provider — either via the IdP's own token endpoint (client credentials or resource-owner-password-grant if permitted for test accounts) or by UI-driving the SSO login exactly once inside globalSetup — capture the resulting cookies/tokens as storageState, and reuse it. If the IdP blocks non-interactive grants for security reasons, I keep one small, tagged suite that does the full UI redirect dance (to catch real SSO regressions), while the bulk of functional tests use the cached session.

Real Project Example On an Azure AD-gated internal analytics tool, the redirect chain (app → Azure AD → MFA prompt → back to app) took 8-12 seconds per test when done live; caching the post-redirect state cut suite time from ~40 minutes to ~6 minutes for a 300-test run, with a separate 5-test “SSO smoke” project running the real flow.

TypeScript Code Example

```
// sso-login.setup.ts – runs once, tagged as a setup project
import { test as setup } from '@playwright/test';

setup('authenticate via Azure AD', async ({ page, context }) => {
  await page.goto('/');
  await page.getByRole('link', { name: 'Sign in with Azure AD' }).click();
  await page.getByLabel('Email').fill(process.env.SSO_USER!);
  await page.getByRole('button', { name: 'Next' }).click();
  await page.getByLabel('Password').fill(process.env.SSO_PASS!);
  await page.getByRole('button', { name: 'Sign in' }).click();
  await page.waitForURL('**/dashboards/**');

  await context.storageState({ path: '.auth/sso-user.json' });
});
```

Common Mistakes Trying to automate MFA push notifications (don’t — use a test tenant with MFA disabled, or a dedicated bypass); re-running the full SSO redirect in every single test “to be safe,” burning enormous CI time for no added coverage.

Follow-Up Questions 1. How would you handle an IdP that rotates session cookies every few minutes? 2. What’s your fallback if the identity provider actively blocks headless browsers? 3. How do you keep the one “real SSO” test from being flaky itself?

QA Architect Tip Say explicitly that you separate “does SSO work” (rare, real, UI-driven) from “does the app work for an authenticated user” (frequent, cached) — conflating these two concerns is the most common design flaw interviewers are probing for.

Q7. How would you design multi-context testing to simulate two users interacting in real time (e.g., a shared dashboard, chat, or collaborative report)?

Difficulty: Senior **Asked In:** Atlassian, Salesforce, ServiceNow **Frequency:** ★★★

Why Interviewer Asks This Real-time collaboration features are common in modern SaaS, and this tests whether you can use contexts as a *testing tool*, not just an isolation mechanism.

Best Interview Answer Because each `BrowserContext` is a fully independent session, I can open two (or more) contexts in the same test — one per simulated user — each with its own `storageState`, and drive both pages within a single test function. This lets me assert cross-user effects deterministically: user A applies a filter or leaves a comment, and I assert user B’s page reflects it, typically after waiting on a `WebSocket` event or a `waitForResponse` rather than a fixed sleep.

Real Project Example On a shared cohort-annotation feature, two analysts viewing the same patient cohort dashboard: analyst A adds a note, and the test asserts analyst B’s page (a second context, same test) shows a live-updated notification badge within a bounded wait, driven by the app’s `WebSocket` push.

TypeScript Code Example

```
import { test, expect } from '@playwright/test';

test('annotation from one analyst appears live for another', async ({ browser }) => {
  const contextA = await browser.newContext({ storageState: '.auth/analyst-a.json' });
  const contextB = await browser.newContext({ storageState: '.auth/analyst-b.json' });
  const pageA = await contextA.newPage();
  const pageB = await contextB.newPage();

  await pageA.goto('/dashboards/cohort/123');
  await pageB.goto('/dashboards/cohort/123');

  await pageA.getByRole('button', { name: 'Add note' }).click();
  await pageA.getByLabel('Note').fill('Re-check exclusion criteria');
  await pageA.getByRole('button', { name: 'Save' }).click();

  await expect(pageB.getByTestId('notification-badge')).toHaveText('1');

  await contextA.close();
  await contextB.close();
});
```

Common Mistakes Using `page.waitForTimeout()` to “give the WebSocket time to arrive” instead of asserting on the actual UI change with Playwright’s auto-retrying `expect`; forgetting to close both contexts, leaking resources across a large parallel run.

Follow-Up Questions 1. How would you simulate three or more concurrent users? 2. How do you assert on a WebSocket message directly rather than the resulting UI change? 3. How would you test a race condition where two users edit the same field simultaneously?

QA Architect Tip Mention that this pattern is also how you test **optimistic locking / conflict resolution** logic — a QA Architect-level interviewer is often listening for whether you’d extend this beyond happy-path collaboration into conflict scenarios.

Q8. What does “dependency injection using fixtures” mean architecturally in Playwright, and why is it a better pattern than helper functions imported into every spec file?

Difficulty: Senior / Staff **Asked In:** Microsoft, EPAM, Thoughtworks **Frequency:** ★★★★★

Why Interviewer Asks This This is the conceptual bridge between “I know Playwright syntax” and “I understand why Playwright Test’s fixture system exists” — a favorite Staff-level differentiator.

Best Interview Answer Playwright’s fixture system is dependency injection: a test declares what it needs (`{ page, apiClient, loggedInAsAdmin }`) as destructured parameters, and the framework resolves and constructs each one lazily, in the right order, sharing scope where declared. This beats plain helper-function imports because fixtures get automatic setup *and* teardown, are composable (fixtures can depend on other fixtures), and are scoped (test vs. worker) — a helper function gives you none of that for free, so you end up hand-writing `beforeEach/afterEach` boilerplate that fixtures replace.

Real Project Example An `apiClient` fixture wraps `APIRequestContext` with base URL and auth headers pre-configured, and a `cohortPage` fixture depends on both `page` and `apiClient`

to seed a test cohort via API before the UI test runs — spec files just ask for cohortPage and get a fully prepared page, with cleanup happening automatically after the test.

TypeScript Code Example

```
// fixtures/index.ts
import { test as base } from '@playwright/test';
import { CohortApiClient } from '../api/CohortApiClient';
import { CohortExplorerPage } from '../pages/dashboards/CohortExplorerPage';

type MyFixtures = {
  apiClient: CohortApiClient;
  cohortExplorerPage: CohortExplorerPage;
};

export const test = base.extend<MyFixtures>({
  apiClient: async ({ request }, use) => {
    const client = new CohortApiClient(request);
    await use(client);
  },

  cohortExplorerPage: async ({ page, apiClient }, use) => {
    const cohortId = await apiClient.createTestCohort({ size: 250 });
    const pageObject = new CohortExplorerPage(page);
    await pageObject.goto(cohortId);

    await use(pageObject);

    await apiClient.deleteTestCohort(cohortId); // teardown, always runs
  },
});

export { expect } from '@playwright/test';

// tests/dashboards/cohort-explorer.spec.ts
import { test, expect } from '../../fixtures';

test('cohort explorer shows correct member count', async ({ cohortExplorerPage }) => {
  await expect(cohortExplorerPage.memberCount).toHaveText('250 members');
});
```

Common Mistakes Writing fixtures that do setup but no teardown (leaking test data); nesting so many fixture dependencies that a single test pulls in expensive setup it doesn't need — fixtures should stay narrow and composable, not turn into a god-object.

Follow-Up Questions 1. How does fixture scope (test vs worker) affect this design? 2. How would you make the apiClient fixture worker-scoped for a read-only client but test-scoped for one that creates data? 3. What happens if fixture A depends on fixture B, and B's setup throws? 4. How do you unit test a fixture itself?

QA Architect Tip Staff candidates explicitly contrast this with the “God BasePage / God helper file” anti-pattern seen in legacy Selenium frameworks, and explain fixtures as the mechanism that keeps large suites maintainable as they scale past a few hundred tests.

SECTION 2 — Advanced Fixtures (Most Asked in 2026)

Q9. Explain test-scoped vs. worker-scoped fixtures, and give a real scenario where getting the scope wrong causes a production-suite bug.

Difficulty: Senior **Asked In:** Microsoft, Walmart Global Tech, PhonePe **Frequency:** ★★★★★

Why Interviewer Asks This Scope mistakes are one of the top causes of subtle cross-test data leakage in real suites — this question separates people who’ve been burned by it from people who haven’t run fixtures at scale.

Best Interview Answer A test-scoped fixture is created fresh for every single test and torn down after — the default, and correct for anything mutable like a page or freshly-created test data. A worker-scoped fixture is created once per worker process and reused across every test that worker runs, correct for expensive, read-only, or naturally shareable resources like a database connection pool or an authenticated API client used only for GET calls. The bug case: if you make a fixture that *creates* a record (like a test cohort) worker-scoped “to save time,” every test in that worker now shares the same cohort, and one test’s mutation (renaming it, deleting a member) silently corrupts the next test’s assumptions — a flaky failure that only appears depending on test execution order within the worker.

Real Project Example A worker-scoped “authenticated API client” fixture is safe and fast to reuse for reading dashboard data across tests; a worker-scoped “seed cohort” fixture caused three engineers to independently debug the same intermittent failure for a week before someone traced it to shared, mutated seed data.

TypeScript Code Example

```
import { test as base } from '@playwright/test';
import { CohortApiClient } from '../api/CohortApiClient';

type Fixtures = {
  readonlyApiClient: CohortApiClient; // safe to share
  freshCohortId: string;              // must NOT be shared
};

export const test = base.extend<Fixtures, { readonlyApiClient: CohortApiClient }>({
  readonlyApiClient: [async ({}, use) => {
    const client = new CohortApiClient();
    await client.authenticate();
    await use(client); // one client per worker, reused across its tests
  }, { scope: 'worker' }],

  freshCohortId: async ({ readonlyApiClient }, use) => {
    const id = await readonlyApiClient.createTestCohort({ size: 100 });
    await use(id); // new cohort per test, deleted after
    await readonlyApiClient.deleteTestCohort(id);
  },
});
```

Common Mistakes Making mutable/creatable resources worker-scoped purely for speed; assuming worker-scoped fixtures run once total (they run once *per worker process*, so with 8 workers you get 8 instances, not 1).

Follow-Up Questions 1. How many times does a worker-scoped fixture’s setup actually run in a 10-worker, 500-test run? 2. Can a worker-scoped fixture depend on a test-scoped one?

What happens? 3. How would you make a database connection pool safely shareable across tests?

QA Architect Tip Frame the decision rule out loud: “read-only and expensive → worker scope; mutable or test-specific → test scope.” Interviewers want to hear the *rule*, not just the definitions.

Q10. What is fixture composition, and how do you build a fixture that depends on multiple other fixtures without creating tangled setup order?

Difficulty: Senior **Asked In:** ServiceNow, Salesforce, EPAM **Frequency:** ★★★★★

Best Interview Answer Fixture composition means one fixture’s setup function destructures other fixtures as its own dependencies — Playwright resolves the dependency graph automatically and initializes each one exactly once per required scope, in the correct order, memoizing results within a test. The way to avoid tangles is discipline: each fixture should have a single clear responsibility, and you should avoid circular or deeply nested chains (more than 2-3 levels deep is usually a sign a fixture is doing too much).

Real Project Example A `loggedInAsAdminWithSeededDashboard` fixture composes three others: `apiClient` (worker-scoped), `adminStorageState` (test-scoped, reads a pre-generated file), and `seededDashboard` (test-scoped, calls the API client to create data) — the spec file asks for one fixture and gets a fully prepared, authenticated, seeded page.

TypeScript Code Example

```
export const test = base.extend<{
  apiClient: CohortApiClient;
  seededDashboardPage: DashboardPage;
}>({
  apiClient: async ({ request }, use) => {
    await use(new CohortApiClient(request));
  },

  seededDashboardPage: async ({ page, apiClient }, use) => {
    const cohortId = await apiClient.createTestCohort({ size: 50 });
    const dashboard = new DashboardPage(page);
    await dashboard.goto(cohortId);
    await use(dashboard);
    await apiClient.deleteTestCohort(cohortId);
  },
});
```

Common Mistakes Building one mega-fixture that does authentication, data seeding, and navigation all inline instead of composing three smaller fixtures — harder to reuse and harder to reason about failures.

Follow-Up Questions 1. How does Playwright avoid re-running a fixture that two other fixtures both depend on? 2. What happens if fixture C depends on both A and B, and A and B each depend on D? 3. How would you unit test just the composition logic?

QA Architect Tip Say explicitly that fixture composition is how you keep spec files declarative — “give me a seeded, authenticated dashboard” — while all imperative detail lives in one well-named, testable place.

Q11. What are “lazy fixtures,” and when should a fixture’s setup cost not be paid unless a test actually uses it?

Difficulty: Senior **Asked In:** Microsoft, Adobe **Frequency:** ★★★

Best Interview Answer Fixtures in Playwright Test are lazy by default — the setup function only runs if a test’s parameter list actually requests that fixture, not for every test in the file. This matters when a fixture is expensive (creating cloud resources, seeding large datasets) and only a subset of tests in a file need it; keeping fixtures request-based rather than blanket `beforeEach` hooks avoids paying that cost for tests that don’t need it.

Real Project Example A `largeCohortDataset` fixture that seeds 10,000 records is only requested by two performance-focused tests in a 40-test file; the other 38 tests never trigger that setup because they don’t declare it as a dependency.

TypeScript Code Example

```
export const test = base.extend<{ largeCohortDataset: string }>({
  largeCohortDataset: async ({ apiClient }, use) => {
    const id = await apiClient.createTestCohort({ size: 10000 }); // expensive
    await use(id);
    await apiClient.deleteTestCohort(id);
  },
});

// Only this test pays the setup cost:
test('pagination performs correctly on very large cohorts', async ({ page, largeCohortData
  // ...
});

// This test never triggers largeCohortDataset setup:
test('empty state renders correctly', async ({ page }) => {
  // ...
});
```

Common Mistakes Converting an expensive, rarely-needed fixture into a `beforeEach` hook “to keep it simple,” which forces every test in the file to pay the cost.

Follow-Up Questions 1. How would you verify a fixture is genuinely lazy in your suite? 2. What’s the trade-off between lazy per-test fixtures and one shared expensive worker-scoped fixture?

QA Architect Tip Connect laziness to CI cost directly — “this is a lever for controlling suite runtime as it grows,” not just a syntax detail.

Q12. Walk through fixture teardown guarantees — what happens if the test body throws an exception mid-test? Does teardown still run?

Difficulty: Senior **Asked In:** Walmart Global Tech, Flipkart, Razorpay **Frequency:** ★★★★★

Best Interview Answer Yes — the code after `await use(...)` in a fixture is guaranteed to run regardless of whether the test passed, failed, or threw, because Playwright wraps fixture teardown similarly to a `try/finally`. This is what makes fixtures safer than manual `afterEach`

hooks for resource cleanup that must always happen, like deleting seeded test data, closing an extra browser context, or releasing a rate-limited API token slot.

Real Project Example A `seededCohort` fixture always deletes the cohort it created in the API's teardown line after `use()`, even when the test assertion above it fails — so a failing test doesn't leave orphaned data that inflates a shared staging environment's record count over thousands of CI runs.

TypeScript Code Example

```
export const test = base.extend<{ seededCohort: string }>({
  seededCohort: async ({ apiClient }, use) => {
    const id = await apiClient.createTestCohort({ size: 20 });
    try {
      await use(id); // test body runs here, may throw
    } finally {
      await apiClient.deleteTestCohort(id); // always runs
    }
  },
});
```

Common Mistakes Assuming teardown is skipped on failure (it isn't) and building redundant manual cleanup elsewhere; not wrapping in `try/finally` when the fixture itself does multi-step setup that could partially fail.

Follow-Up Questions 1. What happens if the teardown code itself throws? 2. How does this interact with `test.afterEach` hooks defined separately in the spec file? 3. How would you guarantee cleanup even if the CI job is killed mid-run (e.g., timeout)?

QA Architect Tip Mention the staging-environment data hygiene angle — architects care about this because orphaned test data from failed runs is a recurring real-world incident, not a theoretical concern.

Q13. How would you design a worker fixture that provides pre-authenticated users for multiple roles, without every worker re-doing the login flow?

Difficulty: Senior / Staff **Asked In:** JP Morgan, Optum, Deloitte Digital **Frequency:** ★★★★★

Best Interview Answer I separate the *acquisition* of auth state (done once in `globalSetup`, writing storage-state files to disk) from the *consumption* of it (a worker-scoped fixture that simply reads the appropriate file based on a role parameter). This means N workers never trigger N logins — they all read from files written once, before any worker starts.

Real Project Example A `roleFixture` worker fixture accepts a role name via `test.use({ role: 'admin' })` per describe block, and resolves to the correct pre-generated storage state file, so a 16-worker run performs exactly the number of logins configured in `globalSetup` — typically 3-4 roles — not 16 or more.

TypeScript Code Example

```
type Role = 'admin' | 'analyst' | 'readOnly';

export const test = base.extend<{}, { role: Role }>({
  role: ['analyst', { option: true, scope: 'worker' }],
});
```

```
storageState: [async ({ role }, use) => {
  await use(`.auth/${role}.json`);
}, { scope: 'worker' }],
});
```

```
// admin-suite.spec.ts
test.use({ role: 'admin' });
test('admin can deactivate a user', async ({ page }) => { /* ... */ });
```

Common Mistakes Running login inside a per-test fixture without caching, effectively re-authenticating on every test; hardcoding one role for the whole suite instead of parameterizing per describe block.

Follow-Up Questions 1. How do you refresh these files if a token expires mid-run on a long regression suite? 2. How would you test role-based access denial (making sure analyst role *cannot* do admin actions)?

QA Architect Tip Explicitly separate “acquire once” vs “consume many times” — this is the general pattern architects apply to any expensive shared setup, not just auth.

Q14. Name two or three fixture anti-patterns you’ve seen in real code-bases, and how you’d refactor them.

Difficulty: Senior / Staff **Asked In:** EPAM, Thoughtworks, Meesho **Frequency:** ★★★

Best Interview Answer First, the “god fixture” — one fixture that logs in, seeds five kinds of data, and navigates three levels deep, used by every test whether it needs all of that or not; refactor by splitting into composable, narrowly-scoped fixtures. Second, “fixture as a dumping ground for assertions” — putting `expect()` calls inside fixture setup instead of the test body, which hides failures behind confusing fixture-setup stack traces; refactor by having fixtures only *produce state*, never assert. Third, “silent fallback fixtures” that swallow errors and return a default value on failure instead of failing loudly, which turns real bugs into false passes downstream.

Real Project Example A legacy suite had a `dashboardWithData` fixture that silently created a *default* cohort if seeding failed instead of throwing — tests kept “passing” against the wrong data for weeks before anyone noticed record counts were off.

TypeScript Code Example

```
// ANTI-PATTERN: swallows errors, returns a fallback silently
seededCohort: async ({ apiClient }, use) => {
  let id;
  try {
    id = await apiClient.createTestCohort({ size: 100 });
  } catch {
    id = 'default-cohort'; // masks real failures
  }
  await use(id);
},
```

```
// REFACTORED: fails loudly, no silent fallback
seededCohort: async ({ apiClient }, use) => {
  const id = await apiClient.createTestCohort({ size: 100 }); // throws on failure, test f
  await use(id);
}
```

```
    await apiClient.deleteTestCohort(id);
  },
```

Common Mistakes Treating “the suite is green” as the only success metric, without checking whether fixtures are hiding real failures behind fallbacks.

Follow-Up Questions 1. How would you detect a silent-fallback fixture already in production via code review? 2. What’s the difference between a fixture retry (acceptable) and a fixture fallback (usually not)?

QA Architect Tip This question is really about *test suite trustworthiness* — Staff candidates should connect anti-patterns to the broader risk of a suite that looks healthy but isn’t actually validating anything.

SECTION 3 — Authentication (Extremely Important)

Q15. Compare API-based login vs. UI-based login for test setup, and explain when you’d deliberately still use UI login.

Difficulty: Senior **Asked In:** Microsoft, Walmart Global Tech, Swiggy **Frequency:** ★★★★★

Best Interview Answer API login hits the auth endpoint directly, captures the returned session/token, and injects it as storageState — fast (milliseconds) and independent of UI changes. UI login drives the actual login form — slower, more fragile to markup changes, but it’s the *only* way to validate that the login page itself works. My rule: 95%+ of tests use API-driven auth for setup speed and stability; a small, explicitly tagged “auth flow” suite still drives the UI login form because that’s a real user journey that needs direct coverage, not just an assumption that the API mechanism implies the UI works too.

Real Project Example On Artemis Health-style platforms, 300+ dashboard tests use storageState from an API login captured once in globalSetup; a separate 8-test auth.spec.ts file exercises the actual login form, invalid-credential messaging, and password-reset link.

TypeScript Code Example

```
// Fast path used by the majority of tests
const context = await request.newContext();
await context.post('/api/auth/login', { data: { username, password } });
await context.storageState({ path: '.auth/analyst.json' });

// Explicit UI-driven auth test — the exception, not the rule
test('shows error on invalid credentials', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Username').fill('wronguser');
  await page.getByLabel('Password').fill('wrongpass');
  await page.getByRole('button', { name: 'Sign in' }).click();
  await expect(page.getByRole('alert')).toHaveText('Invalid username or password');
});
```

Common Mistakes Using UI login everywhere “for realism,” burning 5–10x the CI time for no added coverage once the login flow itself is separately validated.

Follow-Up Questions 1. How do you decide the boundary of what the “auth suite” covers vs. everything else? 2. What if the API login endpoint requires a CSRF token fetched from the UI first?

QA Architect Tip Frame it as risk-based test design: cover the login mechanism directly, and cover *everything else* assuming login works — don't re-verify the same thing 300 times.

Q16. How do you handle token expiry and session timeout scenarios in a long-running regression suite (e.g., 2+ hour run, 1-hour token TTL)?

Difficulty: Senior / Staff **Asked In:** JP Morgan Chase, Optum, ServiceNow **Frequency:** ★★★★★

Best Interview Answer Two separate concerns: (1) tests that *need* a valid session throughout should refresh the token proactively rather than reactively — a worker fixture can check token age before each test and re-authenticate if it's near expiry, before the app itself has a chance to reject a request. (2) Session-expiry *behavior* itself (does the app redirect to login gracefully, does it show a “session expired” toast, does it preserve unsaved form data) needs its own dedicated tests that deliberately force expiry — usually by manipulating the stored token's timestamp or mocking the auth-check API response to return 401.

Real Project Example A 16-worker, 800-test suite with a 1-hour token TTL: I added a worker-scoped fixture that checks token age at the start of each test file and silently refreshes if older than 45 minutes, which eliminated a class of “randomly fails after test 400” flakiness that had nothing to do with the feature under test.

TypeScript Code Example

```
export const test = base.extend<{}, { authManager: AuthManager }>({
  authManager: [async ({}, use) => {
    const manager = new AuthManager();
    await manager.ensureFreshToken(); // refreshes if near expiry
    await use(manager);
  }, { scope: 'worker' }],
});

// Dedicated expiry-behavior test
test('shows session-expired modal and preserves draft', async ({ page }) => {
  await page.goto('/dashboards/cohort-explorer');
  await page.route('**/api/session/check', route =>
    route.fulfill({ status: 401, body: JSON.stringify({ error: 'expired' }) })
  );
  await page.getByLabel('Note').fill('Unsaved draft text');
  await page.getByRole('button', { name: 'Refresh' }).click();
  await expect(page.getByText('Your session expired')).toBeVisible();
  await expect(page.getByLabel('Note')).toHaveValue('Unsaved draft text');
});
```

Common Mistakes Only discovering token-expiry flakiness in CI and “fixing” it with a blanket retry instead of a proactive refresh, which hides the real problem and wastes CI minutes.

Follow-Up Questions 1. How would you simulate expiry deterministically rather than waiting for real time to pass? 2. How does this change for refresh-token rotation flows?

QA Architect Tip Separate “keep my suite stable despite expiry” from “verify the app handles expiry correctly” — conflating them is the most common design flaw here.

Q17. Design a parallel login strategy for a suite where the auth provider rate-limits login attempts.

Difficulty: Staff / QA Architect **Asked In:** Salesforce, Atlassian, Meesho **Frequency:** ★★★

Best Interview Answer Since Playwright workers can start concurrently, naively logging in per-worker or per-test against a rate-limited IdP causes real 429 failures. I fix this by moving all logins into a single sequential `globalSetup` phase (one process, no concurrency) that authenticates every role once *before* any worker starts, writing storage-state files — workers then never call the login endpoint at all, they only read files. If more roles are needed than the rate limit allows in one batch, I stagger logins with small delays inside that single sequential setup, since it's not parallel by nature.

Real Project Example An IdP rate-limited to 5 logins/minute per client ID; sequential `globalSetup` authenticating 4 roles finished well under the limit, whereas the previous “each worker logs in independently” approach with 12 workers regularly triggered 429s at suite start.

TypeScript Code Example

```
async function globalSetup() {
  const roles = ['admin', 'analyst', 'readOnly', 'auditor'];
  for (const role of roles) {
    await loginAndSaveState(role); // sequential, not Promise.all
    await new Promise(r => setTimeout(r, 2000)); // stay under rate limit
  }
}
```

Common Mistakes Using `Promise.all()` across roles in `globalSetup` “for speed,” which reintroduces the exact concurrency problem this design is meant to avoid.

Follow-Up Questions 1. What if a new feature requires a 5th role added mid-project — does this scale? 2. How would you detect rate-limit-related failures vs. genuine auth bugs in CI logs?

QA Architect Tip The insight interviewers want: move concurrency risk *out* of the parallel test phase entirely by concentrating all rate-limited operations into one sequential phase.

Q18. How would you test a multi-tenant application where each tenant has separate SSO configuration?

Difficulty: Staff / QA Architect **Asked In:** ServiceNow, Adobe **Frequency:** ★★★

Best Interview Answer I treat tenant as a first-class dimension alongside role: storage-state files are keyed by tenant-role (e.g., `.auth/acme-admin.json`, `.auth/globex-admin.json`), generated in `globalSetup` for every combination actually needed by the suite, and Playwright projects parameterize which tenant/role pair a given test file runs against — rather than hardcoding tenant assumptions inside individual tests.

Real Project Example A healthcare-analytics platform serving multiple health-system clients, each with its own Okta tenant: cross-tenant isolation tests (does tenant A's admin ever see tenant B's data) are a dedicated, high-priority suite precisely because SSO misconfiguration across tenants is a severe security risk category, not just a functional bug.

TypeScript Code Example

```
projects: [
  { name: 'acme-admin', use: { storageState: '.auth/acme-admin.json' } },
```

```
{ name: 'globex-admin', use: { storageState: '.auth/globex-admin.json' } },  
],
```

Common Mistakes Testing only one “representative” tenant and assuming SSO config differences don’t matter — they frequently do, especially around claim mapping and role assignment.

Follow-Up Questions 1. How would you prioritize which tenants get full regression vs. smoke-only coverage? 2. How do you test that tenant A’s admin truly cannot access tenant B’s data via direct URL manipulation?

QA Architect Tip Elevate cross-tenant isolation to a security-testing conversation, not just a functional one — this is the answer that signals architect-level thinking.

SECTION 4 — Locator Strategy (Modern Playwright)

Q19. Explain locator strict mode and why it’s a deliberate design choice, not a limitation.

Difficulty: Medium / Senior **Asked In:** Microsoft, Walmart Global Tech, EPAM **Frequency:** ★★★★★

Best Interview Answer Strict mode means any action or assertion on a locator that resolves to more than one element throws immediately, instead of silently acting on the first match like older tools did. This is deliberate: silently interacting with “the first match” is exactly how Selenium suites produced tests that passed for the wrong reason when markup changed and a second matching element appeared. Strict mode forces you to write locators precise enough to be unambiguous, which becomes a form of living documentation about the page structure.

Real Project Example A `page.getByRole('button', { name: 'Export' })` locator started failing in strict mode after a new “Export All” button was added to the same panel — correctly surfacing that the original locator was no longer unique, rather than silently clicking the wrong button.

TypeScript Code Example

```
// Throws in strict mode if more than one row matches:  
await page.getByRole('row', { name: 'Cohort A' }).click();  
  
// Fix: scope more precisely  
await page  
  .getByTestId('cohort-table')  
  .getByRole('row', { name: 'Cohort A', exact: true })  
  .click();
```

Common Mistakes Reaching for `.first()` to silence a strict-mode error without checking *why* multiple elements matched — often masking a real duplication bug in the UI.

Follow-Up Questions 1. When is `.first()` legitimately correct vs. a code smell? 2. How does strict mode interact with `locator.count()`?

QA Architect Tip Say that strict-mode failures are usually a *signal*, not noise — investigate the duplication before reaching for `.first()`.

Q20. Why does Playwright recommend getByRole as the top locator priority, and when do you deviate from it?

Difficulty: Senior **Asked In:** Microsoft, Adobe, Atlassian **Frequency:** ★★★★★

Best Interview Answer getByRole matches how assistive technology and real users perceive the page — by accessible role and name — so it's resilient to CSS/class refactors and simultaneously improves accessibility coverage as a side effect. I deviate when the app doesn't expose meaningful roles/names (canvas-based UI, poorly-labeled custom components) or when multiple elements share the same role/name and a data-testid is genuinely the clearer contract between dev and QA for a specific, dynamic component.

Real Project Example Dashboard filter chips rendered as <div> with only a background-color to indicate state have no usable role; I asked the dev team to add aria-pressed so getByRole('button', { pressed: true }) became viable instead of relying on brittle class-name checks.

TypeScript Code Example

```
// Preferred – resilient, accessible
await page.getByRole('button', { name: 'Apply Filters' }).click();

// Fallback for genuinely role-less custom widgets
await page.getByTestId('cohort-heatmap-cell-12-4').click();
```

Common Mistakes Defaulting to data-testid everywhere out of habit, missing the accessibility and resilience benefits getByRole provides for free.

Follow-Up Questions 1. How would you push back on a dev team that resists adding ARIA roles? 2. How do getByRole's name matching rules handle partial text vs exact?

QA Architect Tip Position this as also improving the product's accessibility posture — a strong architect-level answer connects test locator strategy to a11y compliance, a real business concern.

Q21. How do you locate elements inside a virtualized (windowed) table where most rows aren't in the DOM at any given time?

Difficulty: Senior / Staff **Asked In:** Walmart Global Tech, ServiceNow, Flipkart **Frequency:** ★★★★★

Best Interview Answer Virtualized tables (react-window, react-virtualized, ag-Grid virtual scrolling) only render the visible window plus a small buffer — a locator for a row far outside that window simply won't exist yet. The pattern is: scroll the container (not the page) in increments, checking after each scroll whether the target row has entered the DOM, using an auto-retrying expect rather than a single lookup. For assertions on aggregate data (total row count, a specific cell's value), I often prefer asserting via the underlying API response instead of scrolling through thousands of virtual rows in the UI.

Real Project Example A cohort table with 5,000 rows, virtualized to render ~30 at a time: to verify row 3,200 exists with correct data, the test scrolls the grid container programmatically via evaluate (setting scrollTop) in a loop, checking for the target row's test-id after each step, with a max-iteration guard to fail cleanly instead of looping forever.

TypeScript Code Example

```

async function scrollUntilVisible(page: Page, rowTestId: string, maxAttempts = 20) {
  const gridContainer = page.getByTestId('cohort-grid-viewport');
  for (let i = 0; i < maxAttempts; i++) {
    const row = page.getByTestId(rowTestId);
    if (await row.isVisible()) return row;
    await gridContainer.evaluate(el => (el.scrollTop += 300));
    await page.waitForTimeout(50); // brief render settle, not a substitute for the retry
  }
  throw new Error(`Row ${rowTestId} never became visible after ${maxAttempts} scroll attempts`);
}

```

Common Mistakes Trying to assert against all 5,000 rows via the DOM at all, when an API-level assertion is faster, more stable, and tests the same underlying data correctness.

Follow-Up Questions 1. How would you test sort/filter correctness on a virtualized table without scrolling through everything? 2. What's your fallback if the grid library doesn't expose a scrollable container cleanly?

QA Architect Tip Good architects explicitly say “prefer API-level assertion for data correctness, UI-level for rendering behavior” — don't try to make the UI prove something the API already proves better.

Q22. How do you write stable locators into Shadow DOM components, and where does Playwright's piercing behavior stop?

Difficulty: Senior **Asked In:** Microsoft, Salesforce **Frequency:** ★★★

Best Interview Answer Playwright's locators pierce open shadow roots automatically — `page.getByRole()` and `page.locator()` see into shadow DOM without special syntax, unlike old Selenium CSS selectors which stopped at the shadow boundary. This breaks down for *closed* shadow roots, which JavaScript (and therefore Playwright) genuinely cannot inspect from outside — in that case, the only options are asking the dev team to avoid mode: 'closed' for testability, or driving interaction via keyboard/visual coordinates rather than DOM queries.

Real Project Example A design-system `<ds-button>` web component with an open shadow root rendering an inner native `<button>`: `page.getByRole('button', { name: 'Save' })` locates it transparently with no special handling needed.

TypeScript Code Example

```

// Pierces open shadow DOM automatically – no special syntax
await page.getByRole('button', { name: 'Save' }).click();

// If a raw CSS locator is needed across shadow boundaries:
await page.locator('ds-button >> button').click();

```

Common Mistakes Assuming Shadow DOM always needs special Playwright syntax (it usually doesn't for open roots) or giving up entirely on closed roots without first asking the dev team to change the mode.

Follow-Up Questions 1. How would you test a component you truly cannot query into? 2. How does this compare to Selenium's shadow DOM limitations historically?

QA Architect Tip Raise the testability conversation with dev teams proactively — closed shadow roots are usually an accidental choice, not an intentional one, and a quick conversation often resolves it.

Q23. A locator works locally but is flaky in CI only. Walk through your debugging process.

Difficulty: Senior **Asked In:** Walmart Global Tech, Razorpay, Meesho **Frequency:** ★★★★★

Best Interview Answer I start by pulling the CI trace file (trace: 'on-first-retry' or 'retain-on-failure' in config) rather than guessing — the trace shows exact DOM state, network timing, and screenshots at the moment of failure. Common CI-only causes: slower machine timing exposing a real race condition that auto-waiting usually masks locally; a different viewport/headless rendering causing an element to be technically present but not “actionable” (covered by another element, zero-size); or environment-specific data (a shared staging DB with different record counts) making a locator’s expected match count wrong. I reproduce with `--repeat-each` under CI-like resource constraints locally before writing a fix, rather than patching blindly.

Real Project Example A “flaky” export-button locator turned out to be correct — a loading spinner overlay in CI’s slower environment was genuinely still covering the button when the click was attempted; the fix was asserting the spinner was gone first, not adding a retry to the click itself.

TypeScript Code Example

```
// Symptom-fix (masks the real issue):
await page.getByRole('button', { name: 'Export' }).click({ force: true });

// Root-cause fix:
await expect(page.getByTestId('loading-spinner')).toBeHidden();
await page.getByRole('button', { name: 'Export' }).click();
```

Common Mistakes Reaching for `{ force: true }` or blanket retries as the first fix instead of using the trace viewer to find the actual root cause.

Follow-Up Questions 1. What CI-specific config settings (workers, timeouts) would you adjust first when diagnosing this? 2. How do you distinguish “genuinely flaky test” from “genuinely flaky app”?

QA Architect Tip Emphasize trace-first debugging as a non-negotiable discipline — guessing at flaky-test fixes without a trace is a red flag interviewers listen for.

SECTION 5 — Auto Waiting & Synchronization

Q24. Explain Playwright’s actionability checks. What exactly does the framework verify before performing a click?

Difficulty: Senior **Asked In:** Microsoft, Walmart Global Tech, ServiceNow **Frequency:** ★★★★★

Best Interview Answer Before executing an action like `click()`, Playwright waits for the element to be: attached to the DOM, visible, stable (not mid-animation, same bounding box across two consecutive frames), enabled, and — for clicks specifically — that it receives the events (not obscured by another element at that point). This entire pipeline is what “auto-waiting” means, and it’s why Playwright tests need dramatically fewer explicit waits than Selenium

suites did — the framework is already retrying the full actionability check internally up to the action timeout, not just retrying the find.

Real Project Example A “Save” button that’s technically in the DOM immediately but slides into place via a 300ms CSS transition never causes a stale-click failure, because Playwright’s stability check waits for the bounding box to stop changing before clicking.

TypeScript Code Example

```
// No manual wait needed – Playwright's actionability checks handle:  
// attached, visible, stable, enabled, receives-events  
await page.getByRole('button', { name: 'Save' }).click();
```

Common Mistakes Believing auto-waiting means “no waiting is ever needed anywhere” — it only covers actionability of the target element, not business-logic readiness like “has the API response been processed yet.”

Follow-Up Questions 1. Does auto-waiting apply to fill() the same way it applies to click()? 2. What happens if an element becomes actionable then immediately becomes unactionable again?

QA Architect Tip Be precise about the five checks by name — vague answers (“it just waits for the element”) signal shallow familiarity.

Q25. What’s the difference between “actionability” and “business readiness,” and why does confusing them cause flaky tests?

Difficulty: Senior / Staff **Asked In:** Microsoft, Atlassian, Deloitte Digital **Frequency:** ★★★★★

Best Interview Answer Actionability answers “can I physically interact with this DOM element right now” — Playwright handles this automatically. Business readiness answers “has the application finished the operation the user actually cares about” — a button can be perfectly clickable while the underlying data it will act on hasn’t loaded yet. Confusing the two causes tests that click correctly but assert against stale or incomplete data, because nothing forced the test to wait for the *business* event, only the DOM state.

Real Project Example A “Generate Report” button is clickable the instant the page loads, but the cohort data it depends on streams in asynchronously; clicking immediately generates a report against zero records. The fix waits for a business-readiness signal — the cohort count element showing a non-zero value, or a specific API response — before clicking, not just for the button to be enabled.

TypeScript Code Example

```
// Actionability alone – button is clickable but data may not be ready  
await page.getByRole('button', { name: 'Generate Report' }).click();  
  
// Actionability + business readiness  
await expect(page.getByTestId('cohort-count')).not.toHaveText('0 members');  
await page.getByRole('button', { name: 'Generate Report' }).click();
```

Common Mistakes Treating every flaky test as an actionability problem and reaching for `waitForTimeout` instead of identifying the actual business event to wait for.

Follow-Up Questions 1. How would you find the right business-readiness signal for an opaque legacy app? 2. How do web-first assertions help bridge this gap automatically?

QA Architect Tip This distinction is one of the highest-signal answers in the whole interview — architects specifically listen for candidates who can articulate it precisely, because it’s the root cause of most “flaky Playwright test” complaints in practice.

Q26. Why is `page.waitForTimeout()` considered bad practice, and what should replace it in each common use case?

Difficulty: Medium / Senior **Asked In:** Every company in this handbook’s dataset **Frequency:** ★★★★★

Best Interview Answer `waitForTimeout()` is a fixed sleep — it either wastes time waiting longer than necessary (slowing the suite) or doesn’t wait long enough under different load (causing flakiness), and it encodes no information about *what* you’re actually waiting for. Every legitimate use case has a targeted replacement: waiting for navigation → `waitForURL()`; waiting for a network call → `waitForResponse()`; waiting for a load state → `waitForLoadState()`; waiting for UI state → a web-first `expect().toBeVisible()/toHaveText()` which auto-retries. A fixed sleep should only ever appear as a last-resort, clearly-commented workaround for a genuinely unobservable delay, never as a general-purpose fix.

Real Project Example Replacing `await page.waitForTimeout(3000)` after clicking “Export” with `await page.waitForResponse(resp => resp.url().includes('/api/export') && resp.status() === 200)` turned a 3-second-per-test tax with occasional false failures into a sub-second, deterministic wait.

TypeScript Code Example

```
// Bad
await page.getByRole('button', { name: 'Export' }).click();
await page.waitForTimeout(3000);
await expect(page.getByText('Export complete')).toBeVisible();

// Good
await page.getByRole('button', { name: 'Export' }).click();
await page.waitForResponse(resp => resp.url().includes('/api/export') && resp.ok());
await expect(page.getByText('Export complete')).toBeVisible();
```

Common Mistakes “Fixing” flaky tests by increasing the timeout duration instead of switching to an event-based wait — it delays the problem rather than solving it.

Follow-Up Questions 1. Is there ever a legitimate use for `waitForTimeout`? Give a real example. 2. How does `waitForTimeout` interact badly with parallel workers under CPU contention?

QA Architect Tip State plainly: “if I see `waitForTimeout` in a PR review, that’s a request-changes, not a nitpick” — this is a strong, opinionated signal senior interviewers want to hear.

Q27. How do you reliably wait for content rendered via React Suspense / lazy-loaded chunks?

Difficulty: Senior **Asked In:** Microsoft, Adobe, Meesho **Frequency:** ★★★

Best Interview Answer Suspense boundaries typically render a fallback (a spinner, skeleton) while the lazy chunk loads, then swap to real content. The reliable pattern is asserting the fallback disappears *and* the real content’s specific signal appears — not just “wait for the spinner to go away,” because a slow network could show no spinner at all if the check runs

too late, and not just “wait for content” alone in case a stale skeleton with matching test-id lingers.

Real Project Example A lazy-loaded cohort visualization widget shows a skeleton immediately, then the real chart — the test asserts the skeleton’s test-id is hidden AND the chart’s canvas/svg element is visible, guarding against both a race where the skeleton check runs too early and one where the content check runs too late.

TypeScript Code Example

```
await expect(page.getByTestId('cohort-chart-skeleton')).toBeHidden();
await expect(page.getByTestId('cohort-chart-canvas')).toBeVisible();
```

Common Mistakes Only checking for the skeleton’s disappearance and assuming that guarantees the real content is fully rendered — it guarantees the transition started, not that it finished.

Follow-Up Questions 1. How would you handle a Suspense boundary with no visible fallback at all? 2. How does this pattern change for infinite-scroll lazy loading vs. one-time code-splitting?

QA Architect Tip Frame the general principle: assert on the *arrival* of the thing you want, not just the *departure* of the thing you don’t — this generalizes to loaders, modals, and toasts alike.

Q28. How do you test a feature backed by background polling or long-polling, where the UI updates asynchronously without a page navigation?

Difficulty: Senior **Asked In:** ServiceNow, PagerDuty-style incident platforms, Salesforce **Frequency:** ★★★

Best Interview Answer I assert on the *result* of polling — the UI element that changes once new data arrives — using a web-first expect with an extended timeout matching the polling interval, rather than trying to intercept and count individual poll requests unless the test is specifically about polling behavior itself. If the test needs to control timing precisely (e.g., verifying the UI doesn’t update before the *next* poll cycle), I mock the polling endpoint with `page.route()` to return controlled responses on a schedule I define.

Real Project Example A “job status” indicator polls every 5 seconds while a large export job runs; the test mocks three sequential responses (pending, pending, complete) and asserts the UI shows “Export ready” only after the third poll, giving deterministic, fast control over an otherwise slow real-world flow.

TypeScript Code Example

```
let callCount = 0;
await page.route('**/api/export/status', route => {
  callCount++;
  const status = callCount < 3 ? 'pending' : 'complete';
  route.fulfill({ body: JSON.stringify({ status }) });
});

await page.goto('/dashboards/export/123');
await expect(page.getByText('Export ready')).toBeVisible({ timeout: 20000 });
```

Common Mistakes Waiting for a real long-running job to complete in a functional UI test instead of mocking the polling responses, needlessly slowing the suite.

Follow-Up Questions 1. How would you test exponential backoff in a polling implementation? 2. How does this differ when the underlying mechanism is a WebSocket push instead of polling?

QA Architect Tip Mocking the poll schedule to make an inherently slow, real-time feature deterministic and fast is the kind of answer that signals genuine production experience with async UI.

SECTION 6 — Parallel Execution (Top Interview Topic)

Q29. Explain the difference between workers and shards, and when you'd scale one vs. the other.

Difficulty: Senior / Staff **Asked In:** Microsoft, Walmart Global Tech, JP Morgan **Frequency:** ★★★★★

Best Interview Answer Workers are parallel processes *within one machine/CI job*, each running a subset of tests concurrently — controlled by workers in config, bounded by that machine's CPU/memory. Shards split the *entire test suite* across multiple separate CI jobs/machines (`--shard=1/4`), each shard then internally using its own workers. I scale workers up to roughly the CPU core count of the machine (browser automation is CPU-bound, more workers than cores causes contention, not speedup); I scale shards when a single machine's worker ceiling is reached and I need more raw parallelism than one box can offer, distributing across the CI provider's fleet.

Real Project Example An 800-test suite on a 4-core CI runner: 4 workers per shard was the sweet spot locally, and splitting into 4 shards of 200 tests each ($4 \times 4 = 16$ effective parallel lanes across 4 runners) got total wall-clock time down from ~50 minutes to ~8 minutes.

TypeScript Code Example

```
# GitHub Actions matrix – 4 shards, each running with its own workers
strategy:
  matrix:
    shard: [1, 2, 3, 4]
steps:
  - run: npx playwright test --shard=${{ matrix.shard }}/4
```

Common Mistakes Increasing workers far past available CPU cores expecting linear speedup — this usually makes things slower due to context switching and memory pressure.

Follow-Up Questions 1. How do you merge reports across shards into one? 2. How would you decide the optimal shard count for a given CI budget?

QA Architect Tip Tie the answer to cost, not just speed — more shards means more parallel CI minutes billed simultaneously; architects balance wall-clock time against infra cost.

Q30. What is `fullyParallel`, and how does it differ from the default file-level parallelism?

Difficulty: Senior **Asked In:** Microsoft, EPAM, Razorpay **Frequency:** ★★★★★

Best Interview Answer By default, Playwright Test parallelizes at the *file* level — tests within the same file run sequentially in one worker, while different files run across different workers. Setting `fullyParallel: true` makes every individual `test()` — even within the same file — eligible to run in its own worker concurrently. This matters when a single spec file has many independent tests; without it, one heavy file with 40 tests bottlenecks a whole worker regardless of how many workers are configured.

Real Project Example A `dashboard-filters.spec.ts` file with 25 independent filter-combination tests ran sequentially in one worker for 4 minutes even with 8 workers configured; enabling `fullyParallel` distributed those 25 tests across all 8 workers, cutting that file's contribution to ~35 seconds.

TypeScript Code Example

```
export default defineConfig({
  fullyParallel: true,
  workers: 8,
});
```

Common Mistakes Enabling `fullyParallel` without first auditing for shared mutable state between tests in the same file (which file-level sequencing had been accidentally protecting).

Follow-Up Questions 1. What breaks if two tests in the same file share a module-level variable, once `fullyParallel` is on? 2. How does `test.describe.serial` interact with `fullyParallel`?

QA Architect Tip Recommend enabling `fullyParallel` as a deliberate migration step with test-isolation review, not a config flip on day one of a legacy suite.

Q31. When and why would you use `test.describe.serial`, and what's the trade-off?

Difficulty: Senior **Asked In:** ServiceNow, Salesforce, Flipkart **Frequency:** ★★★★★

Best Interview Answer `serial` forces tests within a `describe` block to run in order, in the same worker, and — critically — if one fails, the rest in that block are skipped rather than run against a now-inconsistent state. It's the right tool when tests genuinely depend on prior steps' side effects (a true multi-step workflow like "create draft report → edit it → publish it"), but it's a trade-off: you lose parallelism for that block and lose independent failure signals (one failure masks the rest). I prefer independent tests with API-seeded setup wherever possible, reserving `serial` for workflows that are inherently stateful and expensive to re-seed at every step.

Real Project Example A report-builder workflow (create → add widgets → publish → verify published view) is modeled as one `serial` block of 4 tests rather than one giant test, so failures are still localized to a specific step for debugging, while still enforcing the necessary order.

TypeScript Code Example

```
test.describe.serial('report builder workflow', () => {
  let reportId: string;

  test('creates draft report', async ({ page, apiClient }) => {
    reportId = await apiClient.createDraftReport();
    // ...
  });
});
```

```

test('adds a widget to the draft', async ({ page }) => {
  // uses reportId from previous step
});

test('publishes the report', async ({ page }) => {
  // ...
});
});

```

Common Mistakes Overusing `serial` as a workaround for tests that only *appear* dependent because setup wasn't properly isolated, when API-seeded independent tests would be faster and give clearer failure signals.

Follow-Up Questions 1. How would you refactor a serial block into independent tests if API seeding became available for every step? 2. What happens to test reporting/retries within a serial block on failure?

QA Architect Tip Say explicitly: “serial is a last resort for genuinely stateful workflows, not a shortcut to avoid writing proper setup” — this is the nuance interviewers are checking for.

Q32. Two tests in different files unexpectedly share state and cause flaky parallel failures. How do you diagnose the shared-data source?

Difficulty: Senior / Staff **Asked In:** Walmart Global Tech, Optum, PhonePe **Frequency:** ★★★★★

Best Interview Answer I check, in order: (1) a shared staging/database record both tests read or write (most common — e.g., both tests mutate “the first cohort” in a shared environment); (2) a worker-scoped fixture that’s mutable but was assumed read-only; (3) a module-level variable in a shared helper file that isn’t reset between tests (JS module state persists within a worker process across the files that worker runs); (4) leaked network route mocks from `page.route()` that weren’t scoped correctly. I isolate by running the two suspect tests alone together with `--repeat-each` and comparing worker assignment in the report — if they land on the same worker consistently when failing, that strongly points to in-process shared state over external data collisions.

Real Project Example Two unrelated spec files both called a shared `getFirstCohort()` helper that queried “the most recently created cohort” in a shared staging DB — under parallel execution, whichever test created a cohort last determined what the other test’s “first cohort” resolved to. The fix was making every test create and clean up its own uniquely-named data.

TypeScript Code Example

```

// PROBLEM: relies on shared, mutable external state
async function getFirstCohort() {
  const cohorts = await apiClient.listCohorts();
  return cohorts[0]; // whichever test ran last "wins"
}

// FIX: each test owns its own uniquely-named data
test('cohort filter narrows results', async ({ apiClient }) => {
  const cohortId = await apiClient.createTestCohort({ name: `test-${Date.now()}` });
  // ...
  await apiClient.deleteTestCohort(cohortId);
});

```

Common Mistakes Treating this as “just flaky, add a retry” instead of tracing it to a specific shared resource — retries hide the bug rather than fix it, and it will resurface at higher parallelism.

Follow-Up Questions 1. How would module-level state differ in impact between `fullyParallel: true` and the default file-level parallelism? 2. What CI report signals would tip you off to a worker-sharing issue vs. an external data collision?

QA Architect Tip The senior-level answer treats “which worker did they land on” as a genuine diagnostic data point — most candidates never think to check that.

SECTION 7 — Playwright Configuration (`playwright.config.ts`)

Q33. How do you structure `playwright.config.ts` to support multiple environments (dev, staging, prod-smoke) cleanly?

Difficulty: Senior **Asked In:** Microsoft, Walmart Global Tech, Deloitte Digital **Frequency:** ★★★★★

Best Interview Answer I keep one base config with settings common to all environments, and layer environment-specific overrides via a small set of separate config files (or a single config that reads `process.env.TEST_ENV` and branches on `baseUrl`, `retries`, and `use` options) — never duplicating the full config per environment. Retries and timeouts typically differ meaningfully: 0 retries locally for fast feedback, 1–2 retries in CI against staging to absorb network noise, and a stricter prod-smoke config with tighter timeouts and no destructive-data tests at all.

Real Project Example `playwright.config.ts` reads `baseUrl` from an env-specific `.env` file selected by an npm script (`test:staging`, `test:prod-smoke`), and prod-smoke additionally sets `testDir` to a narrow, explicitly non-destructive subset of tests — never running the full functional suite against production.

TypeScript Code Example

```
import { defineConfig } from '@playwright/test';

const env = process.env.TEST_ENV ?? 'staging';

const envConfig = {
  dev: { baseUrl: 'http://localhost:3000', retries: 0 },
  staging: { baseUrl: 'https://staging.example.com', retries: 1 },
  'prod-smoke': { baseUrl: 'https://app.example.com', retries: 2 },
}[env];

export default defineConfig({
  use: { baseUrl: envConfig.baseUrl },
  retries: envConfig.retries,
  testDir: env === 'prod-smoke' ? './tests/smoke' : './tests',
});
```

Common Mistakes Maintaining three near-duplicate full config files that drift out of sync over time instead of one parameterized base config.

Follow-Up Questions 1. How do you prevent a destructive test from accidentally running against production? 2. How would you handle per-environment feature flags affecting which tests should even run?

QA Architect Tip Explicitly call out a safety mechanism — a `testDir` allowlist or a tag-based filter — that makes it structurally hard to run the full destructive suite against prod by accident.

Q34. Explain how Playwright merges use options across the global config, project-level config, and `test.use()` in a spec file. What wins?

Difficulty: Senior **Asked In:** Microsoft, EPAM, Atlassian **Frequency:** ★★★★★

Best Interview Answer Options merge from broadest to narrowest, with the narrowest scope winning for any key it explicitly sets: global use in the root config is the base, a `projects[].use` block overrides matching keys for that project only, and `test.use()` inside a describe block overrides further for just that block's tests. Keys not overridden at a narrower level simply inherit from the broader scope — it's a shallow merge per key, not a full replace of the whole object.

Real Project Example Global config sets `viewport: { width: 1280, height: 720 }` for all tests; the mobile project overrides `viewport` and `isMobile` for its tests only; a specific describe block within that project further overrides `storageState` to test a different role — each override only touches the key it specifies, everything else still inherits.

TypeScript Code Example

```
// playwright.config.ts – global default
export default defineConfig({
  use: { viewport: { width: 1280, height: 720 }, actionTimeout: 10000 },
  projects: [
    { name: 'mobile', use: { viewport: { width: 375, height: 667 }, isMobile: true } },
  ],
});

// inside a spec file – narrowest override
test.describe('admin flows', () => {
  test.use({ storageState: '.auth/admin.json' });
  // actionTimeout: 10000 and other unset keys still inherited from global
});
```

Common Mistakes Assuming `test.use()` replaces the entire use object rather than merging per key — leading to confusion about why unrelated settings “disappeared.”

Follow-Up Questions 1. How would you override a nested option like `viewport.width` alone without resetting `viewport.height`? 2. Where would you put a setting that should apply to literally every project with no exceptions?

QA Architect Tip Precisely describing “shallow merge, narrowest wins per key” rather than a vague “it overrides” is the differentiator here.

Q35. How do you configure expect timeouts, retries, and snapshot comparison settings differently for a visual-regression-heavy project vs. a functional one?

Difficulty: Senior **Asked In:** Adobe, Salesforce, Meesho **Frequency:** ★★★

Best Interview Answer Functional assertions usually want a moderate `expect.timeout` (5-10s) so genuine bugs fail fast rather than the suite hanging; visual regression tests often need looser pixel-diff thresholds (`maxDiffPixelRatio / threshold` in `toHaveScreenshot`) because anti-aliasing and font-rendering differences between CI and local machines are expected noise, not real failures. I split these into separate projects with different expect config blocks rather than picking one global setting that's wrong for one of the two use cases.

Real Project Example A dashboard visual-regression project sets `toHaveScreenshot({ maxDiffPixelRatio: 0.02 })` to tolerate minor sub-pixel rendering differences across CI runners, while the functional project keeps default strict `toHaveText` matching with no tolerance, since text correctness should never be “approximately right.”

TypeScript Code Example

```
projects: [
  {
    name: 'functional',
    testDir: './tests/functional',
    expect: { timeout: 8000 },
  },
  {
    name: 'visual',
    testDir: './tests/visual',
    expect: { toHaveScreenshot: { maxDiffPixelRatio: 0.02 } },
  },
],
```

Common Mistakes Setting a loose global pixel-diff threshold that accidentally lets real visual regressions in unrelated functional flows slip through unnoticed.

Follow-Up Questions 1. How do you handle visual-regression baseline drift across OS/browser rendering differences? 2. How would you decide which flows deserve visual regression coverage at all vs. just functional?

QA Architect Tip Frame visual regression as a targeted, opt-in tool for specific high-value UI (design-system components, key dashboards) rather than blanket screenshot coverage — over-broad visual suites become high-maintenance noise generators.

SECTION 8 — API Testing (Advanced)

Q36. How do you architect a reusable, typed API client on top of `APIRequestContext` for hybrid API+UI testing?

Difficulty: Senior **Asked In:** Microsoft, Walmart Global Tech, Optum **Frequency:** ★★★★★

Best Interview Answer I wrap `APIRequestContext` in a domain-specific client class (e.g., `CohortApiClient`) exposing typed methods (`createTestCohort()`, `getCohort(id)`, `deleteTestCohort(id)`) rather than letting spec files call `request.post()` with raw URLs

and JSON scattered everywhere. The client owns auth headers, base URL, and response-parsing/error-handling once, so both UI tests (seeding data before driving the browser) and pure API tests (validating the contract directly) share the exact same client — no duplicated request logic between the two test types.

Real Project Example `CohortApiClient.createTestCohort()` is called from UI tests purely for setup speed (skip the UI form to seed data) and from a separate `api/cohort.spec.ts` file that asserts directly on response shape, status codes, and validation error messages — same client, two different purposes.

TypeScript Code Example

```
import { APIRequestContext } from '@playwright/test';

export class CohortApiClient {
  constructor(private request: APIRequestContext) {}

  async createTestCohort(params: { size: number; name?: string }): Promise<string> {
    const response = await this.request.post('/api/cohorts', { data: params });
    if (!response.ok()) throw new Error(`Failed to create cohort: ${response.status()}`);
    const body = await response.json();
    return body.id;
  }

  async deleteTestCohort(id: string): Promise<void> {
    await this.request.delete(`/api/cohorts/${id}`);
  }

  async getCohort(id: string) {
    const response = await this.request.get(`/api/cohorts/${id}`);
    return response.json();
  }
}
```

Common Mistakes Scattering raw `request.post('/api/...')` calls with inline JSON across dozens of spec files — any API contract change then requires editing every file individually instead of one client class.

Follow-Up Questions 1. How would you version this client if the API introduces a v2 endpoint alongside v1? 2. How do you handle auth token refresh transparently inside the client?

QA Architect Tip Present this as the same “single responsibility, reusable abstraction” principle as Page Objects — API clients are the API-testing equivalent, and architects expect you to draw that parallel unprompted.

Q37. How do you test chained/dependent API calls, where call B needs an ID returned by call A?

Difficulty: Senior **Asked In:** JP Morgan, ServiceNow, Razorpay **Frequency:** ★★★★★

Best Interview Answer I write the chain explicitly and sequentially in the test — capture the response of call A, extract the needed field, and use it to construct call B — rather than hardcoding an assumed ID. For complex multi-step chains, I sometimes wrap the whole chain in a helper method on the API client (`createCohortWithMembers()`) so tests read as intent, not plumbing, while the plumbing itself stays testable and reusable.

Real Project Example Creating a cohort, then adding members to it, then generating a report from it, is a three-call chain: cohort ID from call 1 feeds call 2, and the resulting member-count feeds an assertion validated in call 3's response — all driven by real returned IDs, never assumed sequential IDs.

TypeScript Code Example

```
test('report reflects added members', async ({ request }) => {
  const cohortResponse = await request.post('/api/cohorts', { data: { name: 'chain-test' } })
  const { id: cohortId } = await cohortResponse.json();

  await request.post(`/api/cohorts/${cohortId}/members`, {
    data: { memberIds: ['m1', 'm2', 'm3'] },
  });

  const reportResponse = await request.post(`/api/cohorts/${cohortId}/report`);
  const report = await reportResponse.json();

  expect(report.memberCount).toBe(3);
});
```

Common Mistakes Hardcoding IDs assumed from a previous manual run instead of capturing them dynamically from each response — breaks the moment IDs aren't sequential or the environment is reset.

Follow-Up Questions 1. How would you handle a chain where step 2 is eventually-consistent (not immediately available after step 1)? 2. How do you clean up a multi-step chain if step 3 fails partway through?

QA Architect Tip Mention wrapping long chains behind a single semantically-named helper once the pattern repeats across tests — readability at the test level matters as much as correctness.

Q38. How do you validate an API response against a JSON schema instead of asserting individual fields one by one?

Difficulty: Senior **Asked In:** Microsoft, Adobe, Salesforce **Frequency:** ★★★★★

Best Interview Answer For contract-level correctness (right shape, right types, required fields present) I validate against a JSON schema using a library like ajv, rather than manually asserting every field — this scales better as the payload grows and catches unexpected extra/missing fields that field-by-field assertions would silently miss. I still assert specific *values* separately for business-logic correctness (schema validation only proves shape, not that a specific cohort's member count is actually 250).

Real Project Example A cohort API response schema is defined once and reused across every test that touches that endpoint; when a backend change accidentally dropped a required updatedAt field, schema validation caught it immediately across the whole suite instead of one field-check quietly passing because it never checked for that field.

TypeScript Code Example

```
import Ajv from 'ajv';

const ajv = new Ajv();
const cohortSchema = {
```

```

type: 'object',
required: ['id', 'name', 'memberCount', 'updatedAt'],
properties: {
  id: { type: 'string' },
  name: { type: 'string' },
  memberCount: { type: 'number' },
  updatedAt: { type: 'string', format: 'date-time' },
},
};

test('cohort response matches contract schema', async ({ request }) => {
  const response = await request.get('/api/cohorts/123');
  const body = await response.json();

  const validate = ajv.compile(cohortSchema);
  const valid = validate(body);
  expect(valid, JSON.stringify(validate.errors)).toBe(true);
});

```

Common Mistakes Treating schema validation as a full replacement for value assertions — it catches shape drift, not business-logic bugs.

Follow-Up Questions 1. How would you manage schema versioning as the API evolves across breaking changes? 2. How do you share one schema definition between backend contract tests and frontend E2E tests?

QA Architect Tip Position schema validation as “contract testing lite” — a strong bridge answer if the interviewer pushes toward consumer-driven contract testing (Pact) territory.

Q39. How do you handle multipart file upload testing through the API layer, and separately through the UI?

Difficulty: Medium / Senior **Asked In:** Walmart Global Tech, EPAM, Flipkart **Frequency:** ★★★

Best Interview Answer At the API layer, `APIRequestContext.post()` accepts a multipart option directly with file buffers/paths and field names, letting me test upload validation (file type, size limits, malformed files) fast and without touching the browser. At the UI layer, I use `page.setInputFiles()` on the file input, or for drag-and-drop-only upload zones, `page.locator().dispatchEvent('drop', ...)` with a constructed `DataTransfer`-like payload, reserving UI-level upload tests for verifying the actual widget/UX, not for re-testing validation logic already covered at the API level.

Real Project Example A cohort-import CSV upload: API-level tests cover malformed CSV rejection, size limits, and duplicate-header handling quickly; one UI test confirms the drag-and-drop zone and progress indicator work correctly for a valid file.

TypeScript Code Example

```

// API-level – fast validation coverage
test('rejects oversized CSV upload', async ({ request }) => {
  const response = await request.post('/api/cohorts/import', {
    multipart: {
      file: { name: 'huge.csv', mimeType: 'text/csv', buffer: Buffer.alloc(11_000_000) },
    },
  });
});

```

```

    expect(response.status()).toBe(413);
  });

  // UI-level – widget behavior only
  test('shows progress bar during CSV upload', async ({ page }) => {
    await page.goto('/dashboards/import');
    await page.setInputFiles('input[type="file"]', 'test-data/valid-cohort.csv');
    await expect(page.getByRole('progressbar')).toBeVisible();
  });

```

Common Mistakes Re-testing every validation rule (file type, size, malformed content) through the slow UI path when the API layer already covers it comprehensively and faster.

Follow-Up Questions 1. How would you test upload progress percentage accuracy specifically? 2. How do you simulate a network interruption mid-upload?

QA Architect Tip This is a clean example of the general “test the logic where it’s cheapest, test the UX where it lives” principle — say it explicitly.

SECTION 9 — Network Interception & Mocking

Q40. Explain `page.route()` and the difference between `fulfill()`, `abort()`, and `continue()`. When do you use each?

Difficulty: Senior **Asked In:** Microsoft, Walmart Global Tech, Adobe **Frequency:** ★★★★★

Best Interview Answer `page.route()` intercepts matching requests before they reach the network. `fulfill()` responds with a fully custom response, short-circuiting the real backend entirely — used for mocking data, simulating errors, or testing states hard to reproduce naturally (empty states, specific edge-case payloads). `abort()` kills the request outright — used to simulate network failure, blocked third-party scripts, or offline scenarios. `continue()` lets the request proceed to the real network, optionally after modifying headers/params — used when you want to observe or slightly alter a request without fully mocking the response.

Real Project Example Testing a dashboard’s “no data” empty state (a scenario that’s rare in real staging data) is trivial with `fulfill()` returning an empty array; testing graceful degradation when a third-party analytics script fails to load uses `abort()` on that specific request pattern; testing that a client correctly attaches a new required header uses `continue()` with modified headers to confirm the backend accepts it.

TypeScript Code Example

```

// fulfill – full mock
await page.route('**/api/cohorts', route =>
  route.fulfill({ status: 200, body: JSON.stringify([]) })
);

// abort – simulate failure
await page.route('**/analytics.js', route => route.abort());

// continue – pass through with modification
await page.route('**/api/cohorts', route =>
  route.continue({ headers: { ...route.request().headers(), 'x-test-run': 'true' } })
);

```

Common Mistakes Using `abort()` when the intent was actually to mock an error *response* (like a 500) — aborting simulates a network-level failure, not an application-level error response, and the app's error-handling code paths can differ between the two.

Follow-Up Questions 1. How would you simulate a 500 error response specifically vs. a network timeout? 2. How does route matching precedence work if two `page.route()` calls match the same URL pattern?

QA Architect Tip Precisely distinguishing network-level failure (`abort()`) from application-level error response (fulfill with a 4xx/5xx status) is a strong signal — many candidates conflate the two.

Q41. How do you use HAR files for network replay, and when is that better than live `page.route()` mocking?

Difficulty: Senior **Asked In:** Microsoft, ServiceNow, Salesforce **Frequency:** ★★★

Best Interview Answer `page.routeFromHAR()` replays a previously recorded HAR file of real network traffic, giving you a fully deterministic, realistic set of responses without hand-writing every mock — better than manual `page.route()` mocks when you want *realistic* payload shapes across many endpoints at once (e.g., a whole dashboard's worth of API calls) rather than mocking a single targeted endpoint for one specific test scenario. Manual `page.route()` mocking is still better for testing specific edge cases (empty states, errors) that wouldn't naturally appear in a recorded HAR.

Real Project Example Recording a HAR of a fully-loaded dashboard with realistic data lets visual-regression and layout tests run deterministically without depending on a live staging environment's data changing day to day, while targeted functional edge-case tests still use individual `page.route()` mocks.

TypeScript Code Example

```
// Record once (outside the test, via CLI or a setup script):  
// npx playwright open --save-har=dashboard.har --save-har-glob="**/api/**" https://staging.com  
  
test('dashboard renders consistently against recorded data', async ({ page }) => {  
  await page.routeFromHAR('dashboard.har', { url: '**/api/**', update: false });  
  await page.goto('/dashboards/cohort-explorer');  
  await expect(page.getByTestId('cohort-table')).toBeVisible();  
});
```

Common Mistakes Letting HAR files go stale against an evolving API contract without a refresh process — tests then pass against a snapshot of an API shape that no longer matches production.

Follow-Up Questions 1. How would you keep HAR files in sync as the API evolves? 2. What's the risk of committing large HAR files to version control?

QA Architect Tip Mention a periodic HAR-refresh job in CI as part of test-data hygiene — this shows awareness that mocks decay over time if untended.

Q42. How do you test a feature flag that's controlled server-side, without waiting for the actual flag to be toggled in a real environment?

Difficulty: Senior **Asked In:** ServiceNow, Atlassian, Meesho **Frequency:** ★★★★★

Best Interview Answer I mock the feature-flag API/config endpoint response directly via `page.route()`, returning the flag in both “on” and “off” states within the same test file — this lets me validate both code paths deterministically and immediately, without needing an actual flag-management system toggle, an environment-specific flag state, or coordination with the feature-flag team's rollout schedule.

Real Project Example A new cohort-comparison view behind a `cohortComparisonV2` flag: one test mocks the flags endpoint to return `true` and asserts the new UI renders; a second test mocks `false` and asserts the legacy UI still renders correctly — both run in every CI build regardless of the flag's real rollout state.

TypeScript Code Example

```
test('shows new comparison UI when flag is enabled', async ({ page }) => {
  await page.route('**/api/feature-flags', route =>
    route.fulfill({ body: JSON.stringify({ cohortComparisonV2: true }) })
  );
  await page.goto('/dashboards/cohort-explorer');
  await expect(page.getByTestId('comparison-v2-panel')).toBeVisible();
});

test('shows legacy UI when flag is disabled', async ({ page }) => {
  await page.route('**/api/feature-flags', route =>
    route.fulfill({ body: JSON.stringify({ cohortComparisonV2: false }) })
  );
  await page.goto('/dashboards/cohort-explorer');
  await expect(page.getByTestId('comparison-v2-panel')).toBeHidden();
});
```

Common Mistakes Testing only the flag's current real state in the environment and forgetting to cover the other branch — that code path silently goes untested until the flag flips, often right when it matters most.

Follow-Up Questions 1. How would this change for a flag controlled entirely client-side via a config file bundled at build time? 2. How do you avoid flag-mock tests going stale once a flag is fully rolled out and removed from the codebase?

QA Architect Tip Note that flag-mock tests have a natural expiry — part of the process should include removing them once a flag is permanently on/off and the legacy branch is deleted.

Q43. How would you simulate slow or degraded network conditions to test a loading/timeout UX, without actually throttling the real network?

Difficulty: Senior **Asked In:** Adobe, Microsoft, Walmart Global Tech **Frequency:** ★★★

Best Interview Answer Rather than relying on real network throttling (inconsistent across CI runners, hard to reproduce exactly), I delay the mocked response inside `page.route()` using a controlled `await new Promise(r => setTimeout(r, ms))` before calling `fulfill()` or `continue()` — this gives an exact, repeatable delay to test loading spinners, skeleton states, and client-side timeout handling deterministically.

Real Project Example Testing that a dashboard shows a “this is taking longer than usual” message after 8 seconds: the test mocks the relevant API to delay 9 seconds before responding, and asserts the message appears — a scenario that would be extremely slow and flaky to reproduce with real network throttling.

TypeScript Code Example

```
test('shows slow-network message after 8 seconds', async ({ page }) => {
  await page.route('**/api/cohorts', async route => {
    await new Promise(resolve => setTimeout(resolve, 9000));
    await route.fulfill({ body: JSON.stringify([]) });
  });

  await page.goto('/dashboards/cohort-explorer');
  await expect(page.getByText('This is taking longer than usual')).toBeVisible({ timeout:
});
```

Common Mistakes Using Chrome DevTools Protocol network throttling for this kind of precise, assertion-driven timing test — it’s better suited for broad performance investigation than deterministic UX-timeout testing.

Follow-Up Questions 1. How would you test a client-side request timeout that aborts and retries after N seconds? 2. How do you avoid this kind of test needlessly slowing down the whole suite’s runtime?

QA Architect Tip Mention keeping deliberately-slow tests in a separate tagged group so they don’t inflate the default fast-feedback CI run — a suite full of intentional 9-second delays adds up quickly.

SECTION 10 — Debugging & Flaky Test Analysis

Q44. Walk through your systematic process for debugging a test that fails only in CI, never locally.

Difficulty: Senior / Staff **Asked In:** Every company in this handbook’s dataset **Frequency:** ★★★★★

Best Interview Answer First, I make sure CI is configured to capture a trace (trace: 'on-first-retry'), screenshot, and video on failure — without artifacts, debugging is guesswork. Second, I open the trace viewer and walk the exact DOM/network state at the moment of failure, rather than re-reading the test code for typos. Third, I check the usual CI-specific suspects: resource contention (too many workers for CI’s CPU allocation), timing differences (slower machine exposing a real race condition auto-waiting doesn’t fully cover, like business-readiness), and environment/data differences (shared staging state, different seeded data). Fourth, I try to reproduce locally under similar constraints (--workers=1, throttled CPU) before writing a fix, so the fix addresses the actual mechanism, not a guess.

Real Project Example A CI-only failure on an export test traced to CI running 8 workers on a 4-core runner — under real contention, a UI animation took long enough that a business-readiness assertion (waiting for the spinner to disappear) that always passed locally started timing out; the fix was tuning worker count to match CI’s actual core allocation, not increasing the test’s timeout.

TypeScript Code Example

```
export default defineConfig({
  use: {
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
  },
  workers: process.env.CI ? 4 : undefined, // match actual CI core count, not local machine
});
```

Common Mistakes Adding a retry or bumping a timeout as the first response to a CI-only failure without ever opening the trace — this fixes the symptom, and the same class of bug resurfaces at the next scale increase.

Follow-Up Questions 1. How would you tell the difference between a resource-contention issue and a genuine app race condition from a trace alone? 2. What CI config would you check first before even opening a trace?

QA Architect Tip State plainly that “the trace is the primary evidence, everything else is a hypothesis until confirmed” — this discipline is exactly what separates people who patch symptoms from people who fix root causes at scale.

Q45. What information does the Trace Viewer give you that a screenshot alone cannot?

Difficulty: Medium / Senior **Asked In:** Microsoft, Walmart Global Tech, EPAM **Frequency:** ★★★★★

Best Interview Answer A screenshot is one frozen moment. The Trace Viewer gives a full timeline: every action Playwright took with before/after DOM snapshots, the complete network log (requests, responses, timing, headers), console logs, and — critically — the actionability-check reasoning for each action (why a click waited, what it was waiting on). This lets me answer “what was the page’s exact state right before the failing assertion” rather than just “what did it look like at the instant it failed.”

Real Project Example A failing assertion on a cohort count showed “0 members” in the screenshot; the trace’s network tab revealed the actual API response returned 250 members correctly, but a client-side filter defaulting to an unexpected value was hiding them — a bug the screenshot alone couldn’t have distinguished from a genuine backend issue.

TypeScript Code Example

```
npx playwright show-trace trace.zip
```

Common Mistakes Only looking at the final screenshot/video and not walking backward through the action timeline to find where the state actually diverged from expected.

Follow-Up Questions 1. How would you use the trace to debug a flaky click that “sometimes” misses? 2. How does the trace’s network panel help distinguish a backend bug from a frontend bug?

QA Architect Tip Mention that traces are also invaluable evidence when filing a bug against the dev team — attaching a trace file is far more convincing and faster to triage than “it fails sometimes.”

Q46. A test passes when run alone but fails when run as part of the full suite. What’s your hypothesis-driven debugging approach?

Difficulty: Senior / Staff **Asked In:** JP Morgan, Optum, Deloitte Digital **Frequency:** ★★★★★

Best Interview Answer This pattern almost always points to shared state or resource contention introduced by running alongside other tests, not a bug in the test itself. My first hypothesis is a shared external resource (staging data another test also touches); second is worker/CPU contention slowing down actionability or business-readiness timing just enough to tip a marginal wait over the edge; third is genuine test-order dependency (a `beforeAll`/module-level variable another file’s tests mutate). I test each hypothesis directly — run the two suspected conflicting tests together in isolation, run the failing test with `--workers=1` alongside the full suite to rule out contention, and check whether the failure’s timing (early vs. late in the run) correlates with a specific other test file.

Real Project Example A dashboard test that always passed alone failed intermittently in the full suite; isolating it with one other specific file confirmed both tests mutated the same shared “default cohort” record in staging — the actual bug wasn’t in either test’s code but in a missing per-test data isolation strategy.

TypeScript Code Example

```
# Isolate: does it fail alone?
npx playwright test tests/dashboards/cohort-explorer.spec.ts

# Rule out contention: run full suite with 1 worker
npx playwright test --workers=1

# Bisect: run suspected conflicting file pairs together
npx playwright test tests/dashboards/cohort-explorer.spec.ts tests/admin/user-management.s
```

Common Mistakes Assuming “passes alone, fails in the suite” automatically means the test itself is broken, when the actual issue usually lives in shared setup/data, not the test’s own logic.

Follow-Up Questions 1. How would `fullyParallel` change your hypothesis list here? 2. How do you build a permanent guardrail so this class of bug doesn’t recur as the suite grows?

QA Architect Tip Frame the answer as hypothesis-driven, not trial-and-error — naming the three most likely causes upfront before touching any code is exactly the structured thinking Staff-level interviewers are scoring for.

RAPID-FIRE REVISION BANK

Quick-recall coverage for the topics reserved for Volume 2 (full format), organized by theme. Use this for last-day revision — each entry is a concise, interview-ready answer, not the full 9-field breakdown.

Page Object Model & Enterprise Architecture

Q47. What’s the difference between classic Page Object Model and “Component Objects”? ★★★★★ Classic POM models a whole page as one class; Component Objects model reusable UI pieces (a filter panel, a modal, a data table) as their own classes that multiple pages

compose — matches how modern component-based frontends (React/Angular) are actually built, and avoids duplicating logic for shared widgets across pages.

Q48. What should a BasePage class actually contain? ★★★★★ Only truly universal behavior — navigation helpers, common wait utilities, shared header/nav interactions. It should never accumulate feature-specific methods; that’s a sign a shared component object is needed instead.

Q49. How do fixtures and POM work together? ★★★★★ Fixtures handle instantiation, setup, and teardown (including auth and data seeding); Page Objects handle interaction and state exposure. Spec files ask fixtures for a ready-to-use Page Object rather than constructing one manually per test.

Q50. How do you avoid assertions leaking into Page Objects? ★★★★★ Page Objects expose *getters* and *state* (`get memberCount(): Locator`), never `expect()` calls — assertions belong in the test/spec file so failures point clearly to the test’s intent, not buried inside a helper class.

Q51. How do you model a generic reusable Table component object? ★★★★★ Expose methods like `getRowByText()`, `getCellValue(row, column)`, `sortByColumn(name)` parameterized by column/row identifiers, so one Table class serves every data grid in the app instead of a bespoke class per table.

Q52. How do you design a large suite’s test-data/ and utils/ folders to avoid becoming dumping grounds? ★★★ Group test data by domain, not by test file; keep *utils/* limited to pure, stateless functions (formatting, builders) — anything stateful belongs in a *fixture*, not a utility import.

CI/CD

Q53. How do you merge blob reports from multiple shards into one HTML report? ★★★★★ `npx playwright merge-reports --reporter=html ./blob-reports` after each shard uploads its blob report as a CI artifact — gives one unified report instead of N separate ones.

Q54. How do you cache Playwright browser binaries in GitHub Actions to speed up CI? ★★★★★ Cache `~/.cache/ms-playwright` keyed on the Playwright version in `package-lock.json`, and skip `npx playwright install` entirely on a cache hit.

Q55. How do you run Playwright in Docker reliably across different host OS’s? ★★★★★ Use Microsoft’s official `mcr.microsoft.com/playwright` image matching your Playwright version exactly — browser binary/OS dependency mismatches are the most common source of “works on my machine, not in CI” failures.

Q56. What’s a sensible retry strategy for CI vs. local? ★★★★★ 0 retries locally (fast, honest feedback while developing); 1–2 retries in CI to absorb genuine environment noise — but retries should never be raised to mask a consistently flaky test; that test needs fixing, not hiding.

Q57. How do you set up a matrix strategy across browsers and shards simultaneously? ★★★★★ Nest a browser dimension and a shard dimension in the CI matrix (e.g., 3 browsers × 4 shards = 12 parallel jobs), each running `--project=<browser> --shard=<n>/4`.

Performance & Stability

Q58. How do you detect memory leaks across a very long-running test suite? ★★★ Monitor worker process memory over the run; a steadily climbing baseline (not sawtoothing)

across hundreds of tests points to unclosed contexts/pages or accumulating route handlers never removed.

Q59. Should you reuse a browser context across tests for speed? ★★★★★ No — reusing across tests reintroduces state leakage; reuse the *browser process* per worker (already Playwright’s default behavior), not the context.

Q60. How do you tune worker count for a CI runner with limited memory rather than limited CPU? ★★★ Reduce workers even if CPU cores would allow more — each worker’s browser processes consume real memory, and OOM kills manifest as bizarre, unrelated-looking test failures.

Real Coding Interview Challenges (concise prompts)

Q61. Write a fixture-based reusable login that supports two roles. ★★★★★ — see Section 3, Q17 pattern.

Q62. Handle an iframe-embedded payment widget. ★★★★★ `page.frameLocator('iframe[name="payment-number"]').fill(...)` — `frameLocator` scopes all subsequent queries inside that frame automatically.

Q63. Handle a flow that opens a new tab. ★★★★★

```
const [newPage] = await Promise.all([
  context.waitForEvent('page'),
  page.getByRole('link', { name: 'Open in new tab' }).click(),
]);
await newPage.waitForLoadState();
```

Q64. Test file download and verify its contents. ★★★★★

```
const [download] = await Promise.all([
  page.waitForEvent('download'),
  page.getByRole('button', { name: 'Export CSV' }).click(),
]);
const path = await download.path();
```

Q65. Handle an infinite-scroll list to find a specific item. ★★★ Scroll the container in a loop, checking for the target item after each scroll with a max-attempt guard, same pattern as virtualized tables (Section 4, Q21).

Q66. Automate a date-picker calendar widget. ★★★ Prefer typing the date directly into the input if the widget allows it; if not, navigate month-by-month via the widget’s “next month” button and click the specific day cell by accessible name, avoiding brittle grid-coordinate clicks.

Q67. Automate drag-and-drop. ★★★★★ `await source.dragTo(target)` for standard HTML5 drag-drop; for custom JS-based drag libraries that don’t respond to native drag events, use `manual.hover() → mouse.down() → mouse.move() (stepped) → mouse.up()`.

QA Architect Scenario Questions (concise prompts)

Q68. 800 tests take 10 hours — how do you get this under 1 hour? ★★★★★ Layered fix: enable `fullyParallel`, right-size workers to CPU cores, shard across multiple CI machines, move UI-driven setup (login, data seeding) to API calls, audit and eliminate `waitForTimeout()` calls, and split truly independent tests out of unnecessary serial blocks.

Q69. Session expires after 1 hour on an 800-test, multi-hour suite — architecture? ★★★★★ Proactive token-refresh worker fixture checking token age before each test file, plus

a small dedicated suite testing expiry behavior itself (see Section 3, Q16).

Q70. Design a framework for a multi-tenant healthcare application from scratch. ★★★★★ Tenant and role as first-class config dimensions (Section 3, Q18), API-seeded per-tenant test data with strict cleanup, and a dedicated cross-tenant isolation security suite as a first-class citizen, not an afterthought.

Q71. How would you test a microfrontend architecture where different teams own different page regions? ★★★ Contract-test each microfrontend's exposed events/props independently where possible; end-to-end tests validate cross-microfrontend integration points specifically, owned jointly rather than by one team alone, to avoid gaps at the seams.

Q72. Design a regression-selection strategy for a 2,000-test suite where full regression is too slow for every PR. ★★★★★ Tag tests by risk/area, run a fast tagged smoke subset on every PR, full regression on merge to main/nightly, and use change-impact analysis (which files changed → which tagged areas they map to) to selectively expand PR-time coverage.

AI + Playwright (2026)

Q73. How are AI coding assistants (Claude Code, Copilot, Codex) actually used in QA workflows now? ★★★★★ Generating first-draft Page Objects and test skeletons from user stories or PR diffs, auto-suggesting locators from a page snapshot, and summarizing failing-test traces into plain-language root-cause hypotheses — always reviewed by a human before merge, not auto-committed.

Q74. What is MCP (Model Context Protocol) in the context of QA tooling? ★★★★★ A standard way for AI agents to call external tools (Jira, GitHub, Xray, a browser) directly, letting a QA-focused AI agent read a user story, generate a test case, open a PR, and log it in Xray as one connected workflow instead of manual copy-paste between tools.

Q75. What is “AI locator healing,” and what’s the real risk with it? ★★★ Tools that auto-detect a broken locator and suggest/apply a replacement when markup changes. The risk: silent healing can mask a genuine regression that changed an element's role/text for a real reason — healed locators should be flagged for human review, not silently auto-committed.

Q76. How would you use an AI agent for regression-test prioritization? ★★★ Feed it the PR diff and a test-to-code coverage map; it ranks which existing tests are most likely affected, informing which subset to run first for fast feedback — a probabilistic aid, not a replacement for full regression before release.

Company-Specific Emphasis Patterns (from public interview reports, 2025-2026 cycles)

Q77. What does Microsoft tend to emphasize? — Deep Playwright internals (actionability, trace viewer, fixture architecture), given they own the project.

Q78. What does Walmart Global Tech tend to emphasize? — Scale: 1,000+ test suites, sharding/parallelism, CI cost optimization.

Q79. What does JP Morgan Chase tend to emphasize? — Auth/session robustness, security-adjacent testing (SSO, token handling), strict data-hygiene in shared environments.

Q80. What does ServiceNow / Salesforce / Atlassian tend to emphasize? — Multi-tenant SaaS architecture, feature-flag testing, real-time/collaborative feature testing.

Q81. What do EPAM / Thoughtworks / Deloitte Digital tend to emphasize? — Framework design fundamentals and mentoring/code-review scenarios, since these are consulting orgs staffing many client teams.

Q82. What do Flipkart / Swiggy / Razorpay / PhonePe / Meesho tend to emphasize?
— High-throughput, high-parallelism CI at consumer scale, and pragmatic trade-offs under fast release cadence.

End of Volume 1. Volume 2 — full-format deep dives on POM & Enterprise Framework Design, CI/CD, Performance, Coding Challenges, QA Architect Scenarios, AI+Playwright, Company-Specific case studies, and Scenario-Based Debugging (broken-code exercises) — can be generated next in the identical format on request.